Linear Calculi for Programming Languages: a comparison approach

Ana Jorge Almeida, Sandra Alves, and Mário Florido

Faculdade de Ciências, Universidade do Porto, Portugal

Abstract. Here we define and implement a compilation between two linear systems, a linear calculus from David Walker (2005) and Linear Haskell (2017) and show that, although based on different semantics, there exists an equivalence between both systems. There are some difficulties to this though, given that one calculus uses a small-step semantics and the other a big-step semantics and Linear Haskell uses let-statements in addition to the terms that can be directly translated from each system. We will make clear those differences and show how to deal with them in the compilation process.

1 Introduction

A linear function consumes its argument exactly once and a linear type system gives a static correctness guarantee that a function is linear. Linear types originate in a theory of linear logic [7], where truth is a finite resource that can be used once, but not duplicated or discarded.

Several linear calculi were defined before as foundational calculi for programming languages [1, 2, 4–6, 8, 10–12]. In this paper we focus on two of these calculi and compare them with respect to their operational semantics: the linear calculus presented by David Walker in the 2005 chapter *Substructural Type Systems* of the book *Advanced Topics in Programming Languages* [12] and the more hype 2017 Haskell extension *Linear Haskell* [4].

In [12], David Walker presents a linear type system that ensures that we can safely deallocate an object after being used. He also defines an operational semantics that evaluates terms in an abstract machine with an explicit store using a context-based, small-step semantics.

More recently, Linear Haskell, an extension to Haskell that uses linear types [4] is build upon a big-step operational semantics with laziness in the style of Launchbury semantics [9], where terms are transformed before evaluation [4].

Here we define a compilation between these two systems and with this show that, although based on different semantics (small-step and big-step) there exists an equivalence between both systems. As an example, consider the following term of Linear Haskell:

$$\text{let}_\omega\ x : \alpha \to_\omega \alpha = \lambda_\omega y : \alpha.y \text{ in } < x, x >_1$$

where subscripts ω means unrestricted use and 1 linear use. Our compilation translates the previous term to (lin $<$ un $\lambda y :$ un $\alpha.\ y,$ un $\lambda y :$ un $\alpha.\ y >$) in the

David Walker calculus, where *lin* means linear use and *un* means unrestricted use. Below are the evaluations of each term, first in Linear Haskell:

$$\frac{\dfrac{}{(\{x :_\omega = \lambda_\omega y : \alpha.y\}; < x, x >_1) \Downarrow (\{x :_\omega = \lambda_\omega y : \alpha.y\}; < x, x >_1)}}{(\emptyset; \text{let}_\omega\ x : \alpha \to_\omega \alpha = \lambda_\omega y : \alpha.y \text{ in } < x, x >_1) \Downarrow (\{x :_\omega = \lambda_\omega y : \alpha.y\}; < x, x >_1)}$$

and, in the David Walker calculus:

$$\begin{aligned} &(\emptyset;\ \text{lin}\ < \text{un}\ \lambda y : \text{un}\ \alpha.\ y, \text{un}\ \lambda y : \text{un}\ \alpha.\ y >) \\ \to &(\emptyset;\ \text{lin}\ < \text{un}\ \lambda y : \text{un}\ \alpha.\ y, \text{un}\ \lambda y : \text{un}\ \alpha.\ y >) \end{aligned}$$

There are intrinsic difficulties on the definition of a correct compilation, given that one calculus uses a small-step semantics and the other a big-step semantics and Linear Haskell uses let-statements in addition to the terms which can be directly translated from each system. We will make clear those differences and how to deal with them in the compilation process. All definitions were implemented in Haskell and the implementation is available in `https://github.com/anathegrey/Linear-Calculi-Translation/`.

The paper is organised in five sections. In section 2, **David Walker calculus**, we present its syntax, run-time data and operational semantics, and we also provide some necessary definitions, lemmas and propositions relative to this system only. In section 3, **Linear Haskell**, we present its syntax, Launchbury semantics and operational semantics, and some necessary definitions, lemmas and propositions as well. In section 4, **Compilation**, we introduce the translations functions between terms, types, qualifiers and heaps, we talk about some differences between both systems and propose the necessary lemmas that involve both systems, ending with the main proofs. Finally, in section 5, **Conclusion**, we conclude and summarize what was done.

2 David Walker calculus

David Walker proposed a type system that allows for the use of both unrestricted and linear variables. While linear variables are used only once and then discarded, unrestricted variables may be used more than once or not at all [12]. Below, we can see that there are qualifiers, lin if a term is linear and un if it is unrestricted.

Definition 1. *Let us define the calculus's syntax.*

$$\begin{aligned} (qualifiers)\ & q ::= \ lin \mid \ un \\ (terms)\ & t ::= x \mid q < t_1, t_2 > \mid split\ t_1\ as\ x, y\ in\ t_2 \mid q\ \lambda x : T.\ t \\ & \quad \mid \ t_1\ t_2 \\ (prevalues)\ & w ::= < t_1, t_2 > \mid \lambda x : T.t \\ (qualified\ prevalues)\ & v ::= q\ w \\ (stores)\ & S ::= \emptyset \mid S, x \mapsto v \end{aligned}$$

The typechecking of term $q\ \lambda x : T.\ t$ must check that unrestricted functions do not contain linear variables ($q = \text{un}$), which means that an unrestricted function cannot contain a variable that already exists in the context under a linear qualifier [12].

The concepts of free and bound variables follow the normal convention [3].

$$fv(q\ \lambda x : T.\ t) = fv(t) \setminus \{x\}$$
$$bv(\text{split } t_1 \text{ as } x, y \text{ in } t_2) = \{x, y\} \cup bv(t_1) \cup bv(t_2)$$

2.1 Operational Semantics

The operational semantics is factored into two relations. The following relation $(S; t) \rightarrow (S'; t')$ picks out a subcomputation to evaluate, according to the following evaluation contexts, which specify the places in a term where a computation can occur [12].

$$E ::= [\ ] \mid \text{split } E \text{ as } x, y \text{ in } t \mid E\ t$$

The second relation does all the real work. The operational semantics evaluates terms in an abstract machine with an explicit store using a context-based, small-step semantics [12].

$\mathcal{W}$ Operational Semantics:

$$\frac{}{(S; q\ \lambda y : T.t) \rightarrow (S; x \mapsto q\ \lambda y : T.t; x)}\ \textit{s-abs}_{\mathcal{W}}$$

$$\frac{}{(S; q < t_1, t_2 >) \rightarrow (S, x \mapsto q < t_1, t_2 >; x)}\ \textit{s-pair}_{\mathcal{W}}$$

$$\frac{S(a) = q < t_1, t_2 >}{(S; \text{split } a \text{ as } y, z \text{ in } t) \rightarrow (S \sim^q a; t[y \mapsto y_1][z \mapsto z_1])}\ \textit{s-split}_{\mathcal{W}}$$

$$\frac{S(a) = q\ \lambda y : T.t'}{(S; a\ t) \rightarrow (S \sim^q a; t'[y \mapsto t])}\ \textit{s-app}_{\mathcal{W}}$$

In order to avoid creating two sets of operational rules, one for linear data and another for unrestricted data, it was defined an auxiliary function to manage the differences. It works as follows.

$$(S_1, x \mapsto v, S_2) \sim^{\text{lin}} x = S_1, S_2$$
$$S \sim^{\text{un}} x = S$$

An illustrating example follows.

Example 1. Consider

$$\begin{aligned} S \equiv\ & \text{lin } \lambda x :\ \text{lin } (\text{un } (\text{lin } \alpha \rightarrow \text{lin } \alpha)) \rightarrow (\text{lin } (\text{lin } \alpha \rightarrow \text{lin } \alpha)). \\ & \text{lin } \lambda y : \text{lin } (\text{un } (\text{lin } \alpha \rightarrow \text{lin } \alpha)) \rightarrow \text{lin } \alpha. \\ & \text{lin } \lambda z : \text{un } (\text{lin } \alpha \rightarrow \text{lin } \alpha).(x\ z)\ y\ z \end{aligned}$$

We have

$$\frac{}{(\emptyset; S) \rightarrow (a \mapsto S; x)}\ \textit{s-abs}_{\mathcal{W}}$$

2.2 Properties

Here we will prove some properties of David Walker system which are needed for the proof of the main theorems in section 4. The function *deref* (dereferencing) walks in the implementation, through variable references.

Definition 2. *Let* $\boldsymbol{deref_S}$ *be the operation that defines how we remove data from the store, in order to get a term without any free variables.*

$$deref_S(x) = (x, S)$$

$$deref_{S,a\mapsto q\ e}(a) = \begin{cases} deref_{S,a\mapsto q\ e}(e), & \text{if } q = un \\ deref_S(e), & \text{if } q = lin \end{cases}$$

$$\begin{aligned} deref_S(q\ \lambda y : T.t) = {} & let\ (S_1, v) = deref_S(t) \\ & in\ (q\ \lambda y : T.\ v, S_1) \end{aligned}$$

$$\begin{aligned} deref_S(t_1\ t_2) = {} & let\ (v_1, S_1) = deref_S(t_1) \\ & (v_2, S_2) = deref_{S_1}(t_2) \\ & in\ (v_1\ v_2, S_2) \end{aligned}$$

$$\begin{aligned} deref_S(q < t_1, t_2 >) = {} & let\ (v_1, S_1) = deref_S(t_1) \\ & (v_2, S_2) = deref_{S_1}(t_2) \\ & in\ (q < v_1, v_2 >, S_2) \end{aligned}$$

$$\begin{aligned} deref_S(split\ t_1\ as\ y, z\ in\ t_2) = {} & let\ (v_1, S_1) = deref_S(t_1) \\ & (v_2, S_2) = deref_{S_1}(t_2) \\ & in\ (split\ v_1\ as\ y, z\ in\ v_2, S_2) \end{aligned}$$

Definition 3. *The following function defines substitutions on terms.*

$$x[y \mapsto t] = \begin{cases} t, & x = y \\ x, & otherwise \end{cases}$$

$$(q\ \lambda x : q'\ P.\ t_1)[y \mapsto t_2] = \begin{cases} q\ \lambda x : q'\ P.\ t_1, & x = y \\ q\ \lambda x : q'\ P.\ t_1[y \mapsto t_2], & otherwise \end{cases}$$

$$(q < t_1, t_2 >)[y \mapsto t_3] = q < t_1[y \mapsto t_3], t_2[y \mapsto t_3] >$$

$$(t_1\ t_2)[y \mapsto t_3] = t_1[y \mapsto t_3]\ t_2[y \mapsto t_3]$$

$$(split\ t_1\ as\ y, z\ in\ t_2)[x \mapsto t_3] = split\ t_1[x \mapsto t_3]\ as\ y, z\ in\ t_2[x \mapsto t_3]$$

The following lemma, **lemma 1**, expresses that the execution of a term can be split into two parts in order to achieve a normal form, which will help us later when comparing this small-step semantics with the big-step semantics of Linear Haskell. The terms considered are applications and splits and it is proved by induction on the length of the derivation sequences.

Lemma 1.

1. *Given* $(S; t_1\ t_2) \rightarrow_k (S''; v)$*, then* $\exists k_1, k_2 \in \mathbb{N}, k = k_1 + k_2$ *such that*
$$(S; t_1\ t_2) \rightarrow_{k_1} (S'; x\ t_2) \rightarrow_{k_2} (S''; v)$$
2. *Given* $(S; split\ t_1\ as\ y, z\ in\ t_2) \rightarrow_k (S''; v)$*, then* $\exists k_1, k_2 \in \mathbb{N}, k = k_1 + k_2$ *such that:*
$$(S; split\ t_1\ as\ y, z\ in\ t_2) \rightarrow_{k_1} (S'; split\ x\ as\ y, z\ in\ t_2) \rightarrow_{k_2} (S''; v)$$

Proposition 1 allows us to remove variables from the store using function *deref*, regardless of their defining qualifier.

Proposition 1. *If* $a \notin fv(t)$ *then* $deref_{S, a \mapsto q\ e}(t) = deref_S(t)$.

The $deref_S$ **lemma** formalises the notion that, when a variable occurs in the store, then it will have to be substituted in the term. This is proved by induction on the terms and uses **Proposition 1**.

Lemma 2 ($deref_S$ **lemma).**
Given $S, a \mapsto q\ e$*, then* $deref_{S, a \mapsto q\ e}(t) = deref_S(t[a \mapsto e])$.

Finally, the S **substitution lemma** proves, by induction on the lenght of the derivation tree, that if a variable occurs in the store then we can remove it from the store and substitute it directly on the term, and after applying the *deref* function, the result of both computations is the same.

Lemma 3 (S **Substitution Lemma).**
If $(S, a \mapsto q\ t; M) \twoheadrightarrow (S_1; N_1)$ *then*
$$(S; M[a \mapsto t]) \twoheadrightarrow (S_2; N_2) \text{ and } deref_{S_2}(N_2) = deref_{S_1}(N_1)$$

3 Linear Haskell

Just as in David Walker's system, Linear Haskell uses both unrestricted and linear variables, qualifying them by using multiplicities ω and 1 respectively, thus Linear Haskell extends Haskell with linear types [4] and has both a linear arrow to introduce linear objects in the environment, and an unrestricted arrow to introduce unrestricted objects [4].

We have adapted some terms to be consistent with the David Walker calculus, using pairs and split, which are special cases of the data constructors and case-expressions presented in Linear Haskell.

Definition 4. *The syntax of the term language is as follows.*

$$\begin{array}{rrl}
(multiplicities) & \pi ::= & 1 \mid \omega \\
(terms) & t, e_i ::= & x \mid < t_1, t_2 >_\pi \mid split\ t_1\ as\ x, y\ in\ t_2 \mid \lambda_\pi x : A.\ t \\
 & & \mid\ t_1\ t_2 \mid\ let_\pi\ a_1 : A_1 = e_1, \ldots, a_n : A_n = e_n\ in\ t \\
(environment) & \Gamma, \Delta ::= & \emptyset \mid x :_\pi A = t, \Gamma
\end{array}$$

Multiplicities are denoted by the π subscript and in the type arrows, where ω means unrestricted use and 1 linear use. The concepts of free and bound variables follow the normal convention [3].

$$bv(\lambda_\pi x : A.t) = \{x\} \cup bv(t)$$
$$fv(\text{split } t_1 \text{ as } x, y \text{ in } t_2) = (fv(t_1) \cup fv(t_2)) \setminus \{x, y\}$$

3.1 Operational Semantics

Linear Haskell uses a big-step operational semantics with laziness in the style of Launchbury semantics for lazy evaluation [4]. This means that terms are transformed ahead of execution in order to reflect the amount of sharing that a language like Haskell would offer [4].

Definition 5. *The syntax for the Launchbury-style semantics is as follows.*

$$\begin{aligned}
(\lambda_\pi y : A.\ t)^* &= \lambda_\pi y : A.\ t^* \\
y^* &= y \\
(t\ y)^* &= t^*\ y \\
(t\ u)^* &= let_\pi\ a : A = u^*\ in\ t^*\ a \\
&\quad because\ \Gamma \vdash t : A \to_\pi B \\
(< x_1, x_2 >_\pi)^* &= < x_1, x_2 >_\pi \\
(< t_1, t_2 >_\pi)^* &= let_\pi\ a_1 : A_1 = {t_1}^*, a_2 : A_2 = {t_2}^* \\
&\quad in\ < a_1, a_2 >_\pi \\
(split\ t_1\ as\ y, z\ in\ t_2)^* &= split\ {t_1}^*\ as\ x, y\ in\ {t_2}^* \\
(let_\pi\ a_1 : A_1 = e_1, \ldots, a_n : A_n = e_n\ in\ e)^* &= let_\pi\ a_1 : A_1 = {e_1}^*, \ldots, a_n : A_n = {e_n}^* \\
&\quad in\ e^*
\end{aligned}$$

Let us consider the following example.

Example 2. Given term $M \equiv\ (\lambda_1 x : \alpha \to_1 \alpha.\ x)\ (\lambda_1 y : \alpha.\ y)$.
Now we have

$$\begin{aligned}
M^* &= \text{let}_1\ a : \alpha \to_1 \alpha = (\lambda_1 y : \alpha.\ y)^*\ \text{in}\ (\lambda_1 x : \alpha \to_1 \alpha.\ x)^*\ a \\
&= \text{let}_1\ a : \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y^*\ \text{in}\ (\lambda_1 x : \alpha \to_1 \alpha.\ x^*)\ a \\
&= \text{let}_1\ a : \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y\ \text{in}\ (\lambda_1 x : \alpha \to_1 \alpha.\ x)\ a
\end{aligned}$$

After the transformation of terms, they are evaluated. Here, the type annotation is kept in the operational semantics of Linear Haskell.

$\mathcal{L}$ Operational Semantics:

$$\frac{}{(\Gamma; \lambda_\pi x : A.\ e) \Downarrow (\Gamma; \lambda_\pi x : A.\ e)}\ \textit{s-abs}_{\mathcal{L}}$$

$$\frac{}{(\Gamma; < x, y >_\pi \Downarrow (\Gamma; < x, y >_\pi)}\ \textit{s-pair}_{\mathcal{L}}$$

$$\frac{(\Gamma; e) \Downarrow (\Delta; < x_1, y_1 >_\pi) \qquad (\Delta; t[x \mapsto x_1][y \mapsto y_1]) \Downarrow (\Theta; z)}{(\Gamma; \text{split}\ e\ \text{as}\ x, y\ \text{in}\ t) \Downarrow (\Theta; z)}\ \textit{s-split}_{\mathcal{L}}$$

$$\frac{(\Gamma; e) \Downarrow (\Delta; \lambda_\pi y : A.\ e') \qquad (\Delta; e'[y \mapsto x]) \Downarrow (\Theta; z)}{(\Gamma, e\ x) \Downarrow (\Theta; z)}\ \textit{s-app}_{\mathcal{L}}$$

$$\frac{(\Gamma; e) \Downarrow (\Delta; z)}{(\Gamma, a :_\omega A = e; a) \Downarrow (\Delta, a :_\omega A = z; z)}\ \textit{s-unvar}_{\mathcal{L}}$$

$$\frac{(\Gamma; e) \Downarrow (\Delta; z)}{(\Gamma, a :_1 A = e; a) \Downarrow (\Delta; z)}\ \textit{s-linvar}_{\mathcal{L}}$$

$$\frac{(\Gamma, x_1 :_\pi A_1 = e_1, \ldots, x_n :_\pi A_n = e_n; e) \Downarrow (\Delta; z)}{(\Gamma; \mathrm{let}_\pi\ x_1 : A_1 = e_1, \ldots, x_n : A_n = e_n \text{ in } e \Downarrow (\Delta; z)}\ \textit{s-let}_{\mathcal{L}}$$

Let us use the same term as in example 2 as it is already transformed.

Example 3. Consider Ω_1, which is by rule *s-abs*$_{\mathcal{L}}$.

$$(\{a :_1 \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y\}; \lambda_1 x : \alpha \to_1 \alpha.\ x)$$
$$\Downarrow (\{a :_1 \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y\}; \lambda_1 x : \alpha \to_1 \alpha.\ x)$$

And let us consider Ω_2.

$$\frac{\dfrac{}{(\emptyset; \lambda_1 y : \alpha.\ y) \Downarrow (\emptyset; \lambda_1 y : \alpha.\ y)}\ \textit{s-abs}_{\mathcal{L}}}{(\{a :_1 \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y\}; a) \Downarrow (\emptyset; \lambda_1 y : \alpha.\ y)}\ \textit{s-linvar}_{\mathcal{L}}$$

$$\frac{\dfrac{\Omega_1 \qquad \Omega_2}{(\{a :_1 \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y\}; (\lambda_1 x : \alpha \to_1 \alpha.\ x)\ a) \Downarrow (\emptyset; \lambda_1 y : \alpha.\ y)}\ \textit{s-app}_{\mathcal{L}}}{(; \mathrm{let}_1\ a : \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y \text{ in } (\lambda_1 x : \alpha \to_1 \alpha.\ x)\ a) \Downarrow (\emptyset; \lambda_1 y : \alpha.\ y)}\ \textit{s-let}_{\mathcal{L}}$$

3.2 Properties

Here we also need to define some new notions, which will be used in the proofs of the main Theorems in section 4.

Definition 6. *Let* $\boldsymbol{deref_\Gamma}$ *be the operation that defines how we remove data from the environment, in order to get a term without any free variables.*

$$\begin{aligned}
&deref_\Gamma(x) = (x, \Gamma)\\
&deref_{\Gamma, a:_\pi A = e}(a) = \begin{cases} deref_{\Gamma, a:_\pi A = e}(e), & \text{if } \pi = \omega \\ deref_\Gamma(e), & \text{if } \pi = 1 \end{cases}\\
&deref_\Gamma(\lambda_\pi y : P.\ t) = \textit{let } (v, \Gamma_1) = deref_\Gamma(t)\\
&\qquad\qquad \textit{in } (\lambda_\pi y : P.\ v, \Gamma_1)\\
&deref_\Gamma(t_1\ t_2) = \textit{let } (v_1, \Gamma_1) = deref_\Gamma(t_1)\\
&\qquad\qquad (v_2, \Gamma_2) = deref_{\Gamma_1}(t_2)\\
&\qquad\qquad \textit{in } (v_1\ v_2, \Gamma_2)\\
&deref_\Gamma(< t_1, t_2 >_\pi) = \textit{let } (v_1, \Gamma_1) = deref_\Gamma(t_1)\\
&\qquad\qquad (v_2, \Gamma_2) = deref_{\Gamma_1}(t_2)\\
&\qquad\qquad \textit{in} (< v_1, v_2 >_\pi, \Gamma_2)\\
&deref_\Gamma(\textit{split } t_1 \textit{ as } y, z \textit{ in } t_2) = \textit{let } (v_1, \Gamma_1) = deref_\Gamma(t_1)\\
&\qquad\qquad (v_2, \Gamma_2) = deref_{\Gamma_1}(t_2)\\
&\qquad\qquad \textit{in } (\textit{split } v_1 \textit{ as } y, z \textit{ in } v_2, \Gamma_2)\\
&deref_\Gamma(\textit{let}_\pi\ a_1 : A_1 = e_1, \ldots, a_n : A_n = e_n \textit{ in } t) = deref_{\Gamma, a_1:_\pi A_1 = e_1, \ldots, a_n:_\pi A_n = e_n}(t)
\end{aligned}$$

Definition 7. *The following function defines substitution on terms.*

$$x[y \mapsto t_2] = \begin{cases} t_2, & x = y \\ x, & otherwise \end{cases}$$

$$(\lambda_\pi x : A.\ t_1)[y \mapsto t_2] = \begin{cases} \lambda_\pi x : A.\ t_1, & x = y \\ \lambda_\pi x : A.\ t_1[y \mapsto t_2], & otherwise \end{cases}$$

$$(< t_1, t_2 >_\pi)[y \mapsto t_3] =< t_1[y \mapsto t_3], t_2[y \mapsto t_3] >_\pi$$

$$(t_1\ t_2)[y \mapsto t_3] = t_1[y \mapsto t_3]\ t_2[y \mapsto t_3]$$

$$(split\ t_1\ as\ y, z\ in\ t_2)[x \mapsto t_3] = split\ t_1[x \mapsto t_3]\ as\ y, z\ in\ t_2[x \mapsto t_3]$$

$$(let_\pi\ a_1 : A_1 = e_1, \ldots, a_n : A_n = e_n\ in\ t)[y \mapsto t']$$
$$= (let_\pi\ a_1 : A_1 = e_1[y \mapsto t'], \ldots, a_n : A_n = e_n[y \mapsto t']\ in\ t[y \mapsto t'])$$

We now prove some auxiliary lemmas, which will also be used in the proofs of our main theorems.

Proposition 2 shows that if a variable occurs in the environment but does not occur free in the term, then its presence has no influence in the result of the evaluation.

Proposition 2.
Given $x \in \Gamma$, if $(\Gamma; t) \Downarrow (\Gamma'; u)$ and $x \notin fv(t)$ then $(\Gamma \setminus x; t) \Downarrow (\Gamma' \setminus x; u)$.

Proposition 3 shows that, if there is a substitution on the term where a variable is substituted by another existing in the environment then, by applying the *deref* function, it is equal to substituting that variable on the term by the value saved in the environment.

Proposition 3.
Given $(\Gamma, a :_\pi A = t_2; t_1[y \mapsto a])$, where $a \notin fv(t_1)$, then

$$deref_{\Gamma, a :_\pi A = t_2}(t_1[y \mapsto a]) = deref_\Gamma(t_1[y \mapsto t_2])$$

Proposition 4 allows us to remove variables from the environment using function deref, regardless of their defining qualifier, because there is no use in maintaining a variable that does not occur free in the term.

Proposition 4. *If $a \notin fv(t)$ then $deref_{\Gamma, a :_\pi A = e}(t) = deref_\Gamma(t)$.*

The $deref_\Gamma$ **lemma** formalises the notion that, whenever a variable occurs in the environment, then it will have to be substituted in the term. This is proved by induction on the terms and uses **proposition 4**.

Lemma 4 ($deref_\Gamma$ lemma).
Given $\Gamma, a :_\pi A = e$, then $deref_{\Gamma, a :_\pi A = e}(t) = deref_\Gamma(t[a \mapsto e])$.

Lemma 5 proves, by induction on the length of the derivation tree, that if a variable does not occur free in the term but it needs to substitute another variable in values saved in the environment, then we can perform that substitution using directly the value in the environment under that first variable and, by applying the *deref* function, both computations will yield the same result.

Lemma 5.
Given that $x \notin fv(t)$*, if* $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)$ *then*

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)$$

$$\textit{and } deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$$

Finally, the Γ **substitituion lemma** proves, by induction on the length of the derivation tree, that if a variable occurs in the environment and we perform a substitution with that variable, then we can remove it from the environment and substitute it directly on the term and, after applying the *deref* function, the result of both computations will yield the same result.

Lemma 6 (Γ Substitution Lemma).
If $(\Gamma, a :_{\pi} A = t_2; t_1[y \mapsto a]) \Downarrow (\Gamma_1; \sigma_1)$ *then*

$$(\Gamma; t_1[y \mapsto t_2]) \Downarrow (\Gamma_2; \sigma_2) \textit{ and } deref_{\Gamma_1}(\sigma_1) = deref_{\Gamma_2}(\sigma_2)$$

4 Compilation

Our main goal is to define a translation to show that both systems are equivalent. Therefore, we first define translations between terms, types, qualifiers and heaps. Below are the definitions.

Definition 8. *Translation of terms from* $\mathcal{W}$ *to* $\mathcal{L}$*.*
$x_{\mathcal{L}} = x$
$(q < t_1, t_2 >)_{\mathcal{L}} =< t_{1\mathcal{L}}, t_{2\mathcal{L}} >_{q_{\mathcal{L}}}$
$(split\ t_1\ as\ x, y\ in\ t_2)_{\mathcal{L}} = split\ t_{1\mathcal{L}}\ as\ x, y\ in\ t_{2\mathcal{L}}$
$(q\ \lambda x : q'\ P.\ t)_{\mathcal{L}} = \lambda_{q'_{\mathcal{L}}} x : P_{\mathcal{L}}.\ t_{\mathcal{L}}$
$(t_1\ t_2)_{\mathcal{L}} = t_{1\mathcal{L}}\ t_{2\mathcal{L}}$

Definition 9. *Translation of terms from* $\mathcal{L}$ *to* $\mathcal{W}$*.*
$x_{\mathcal{W}} = x$
$(< t_1, t_2 >_{\pi})_{\mathcal{W}} = \pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >$
$(split\ t_1\ as\ x, y\ in\ t_2)_{\mathcal{W}} = split\ t_{1\mathcal{W}}\ as\ x, y\ in\ t_{2\mathcal{W}}$
$(\lambda_{\pi} x : P.\ t)_{\mathcal{W}} = \pi_{\mathcal{W}}\ \lambda x : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}}$
$(t_1\ t_2)_{\mathcal{W}} = t_{1\mathcal{W}}\ t_{2\mathcal{W}}$
$(let_{\pi}\ a_1 : A_1 = e_1, \ldots, a_n : A_n = e_n\ in\ t)_{\mathcal{W}} = t_{\mathcal{W}}[a_1 \mapsto e_{1\mathcal{W}}] \ldots [a_n \mapsto e_n \mathcal{W}]$

Definition 10. *Translation of S:* $(a = q\ t, S)_{\mathcal{L}} = (a :_{q_{\mathcal{L}}} P = t_{\mathcal{L}}, S_{\mathcal{L}})$*, where* $\Gamma' \vdash t_{\mathcal{L}} : P$

Definition 11. *Translation of* Γ*:* $(a :_{\pi} A = t, \Gamma)_{\mathcal{W}} = (a = \pi_{\mathcal{W}}\ t_{\mathcal{W}}, \Gamma_{\mathcal{W}})$

Definition 12. *Translation of types.*

Translation of types from $\mathcal{W}$ to $\mathcal{L}$:

$q\ (q'\ P_1 \to q''\ P_2)_{\mathcal{L}} = P_{1\mathcal{L}} \to_{q'\mathcal{L}} P_{2\mathcal{L}}$

$q\ (T_1 * T_2) = T_{1\mathcal{L}} *_{q\mathcal{L}} T_{2\mathcal{L}}$

Translation of types from $\mathcal{L}$ to $\mathcal{W}$:

$(T_1 \to_\pi T_2)_{\mathcal{W}} = \pi_{\mathcal{W}}\ T_{1\mathcal{W}} \to \pi_{\mathcal{W}}\ T_{2\mathcal{W}}$

$(T_1 *_\pi T_2)_{\mathcal{W}} = \pi_{\mathcal{W}}\ (T_{1\mathcal{W}} * T_{2\mathcal{W}})$

Definition 13. *Translation of qualifiers.*

$lin_{\mathcal{L}} = 1$

$un_{\mathcal{L}} = \omega$

Definition 14. *Translation of multiplicities.*

$1_{\mathcal{W}} = lin$

$\omega_{\mathcal{W}} = un$

Consider the following examples.

Example 4. To translate a $\mathcal{W}$-term into an $\mathcal{L}$-term, let us consider

$$\begin{aligned} S \equiv\ & \text{lin } \lambda x :\ \text{lin } (\text{un } (\text{lin } \alpha \to \text{lin } \alpha)) \to (\text{lin } (\text{lin } \alpha \to \text{lin } \alpha)). \\ & \text{lin } \lambda y : \text{lin } (\text{un } (\text{lin } \alpha \to \text{lin } \alpha)) \to \text{lin } \alpha. \\ & \text{lin } \lambda z : \text{un } (\text{lin } \alpha \to \text{lin } \alpha).x\ z\ (y\ z) \end{aligned}$$

Now we have

$$\begin{aligned} S_{\mathcal{L}} &= \lambda_1 x : (\alpha \to_1 \alpha) \to_1 (\alpha \to_1 \alpha).\ \lambda_1 y : (\alpha \to_1 \alpha) \to_1 \alpha.\ \lambda_\omega z : \alpha \to_1 \alpha.(x\ z\ (y\ z))_{\mathcal{L}} \\ &= \lambda_1 x : (\alpha \to_1 \alpha) \to_1 (\alpha \to_1 \alpha).\ \lambda_1 y : (\alpha \to_1 \alpha) \to_1 \alpha.\ \lambda_\omega z : \alpha \to_1 \alpha.x\ z\ (y\ z) \end{aligned}$$

Example 5. To translate an $\mathcal{L}$-term into a $\mathcal{W}$-term, let us consider

$$M^* \equiv\ \text{let}_1\ a : \alpha \to_1 \alpha = \lambda_1 y : \alpha.\ y \text{ in } (\lambda_1 x : \alpha \to_1 \alpha.\ x)\ a$$

Now we have

$$\begin{aligned} (M^*)_{\mathcal{W}} &= ((\lambda_1 x : \alpha \to_1 \alpha.\ x)\ a)_{\mathcal{W}}[a \mapsto (\lambda_1 y : \alpha.\ y)_{\mathcal{W}}] \\ &= ((\text{lin } \lambda x : \text{lin } (\text{lin } \alpha \to \text{lin } \alpha).\ x)\ a)[a \mapsto \text{lin } \lambda y : \text{lin } \alpha.\ y] \\ &= (\text{lin } \lambda x : \text{lin } (\text{lin } \alpha \to \text{lin } \alpha).\ x)\ (\text{lin } \lambda y : \text{lin } \alpha.\ y) \end{aligned}$$

We know that, on David Walker's system,

$deref_S(q\ \lambda y : q'\ P.\ t) = q\ \lambda y : q'\ P.\ t$ and $deref_S(q < t_1, t_2 >) = q < t_1, t_2 >$ because abstractions and pairs are not evaluated, so they will not have any address variables.

An illustrating example follows.

Example 6. By applying rule *s-abs*$_{\mathcal{W}}$, we have

$$\begin{aligned} & (\emptyset; \text{lin } \lambda f : \text{un } (\text{lin } \alpha \to\ \text{lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha.f\ (f\ x)) \\ \to\ & (a \mapsto \text{lin } \lambda f : \text{un } (\text{lin } \alpha \to\ \text{lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha.f\ (f\ x); a) \end{aligned}$$

We have

$$\begin{aligned} & deref_{a \mapsto \text{lin } \lambda f : \text{un } (\text{lin } \alpha \to\ \text{lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha.f\ (f\ x)}(a) \\ =\ & deref_\emptyset(\text{lin } \lambda f : \text{un } (\text{lin } \alpha \to\ \text{lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha.f\ (f\ x)) \\ =\ & \text{lin } \lambda f : \text{un } (\text{lin } \alpha \to\ \text{lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha.f\ (f\ x) \end{aligned}$$

But the same does not happen on Linear Haskell. Let us use the same example.

Example 7.

$$\begin{aligned}
&(\text{lin } \lambda f : \text{un } (\text{lin } \alpha \to \text{ lin } \alpha).\ \text{lin } \lambda x : \text{lin } \alpha. f\ (f\ x))_{\mathcal{L}}{}^{*} \\
&= (\lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha. (f\ (f\ x)))^{*} \\
&= \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha.\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1
\end{aligned}$$

By rule *s-abs*$_{\mathcal{L}}$,

$$\begin{aligned}
&(\emptyset; \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha.\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1) \\
\Downarrow\ &(\emptyset; \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha.\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1)
\end{aligned}$$

We have

$$\begin{aligned}
&\mathit{deref}_{\emptyset}(\lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha.\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1) \\
&= \lambda_\omega f : \alpha \to_1 \alpha. \mathit{deref}_{\emptyset}(\lambda_1 x : \alpha.\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1) \\
&= \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha. \mathit{deref}_{\emptyset}(\ \text{let}_1\ a_1 : \alpha = f\ x \text{ in } f\ a_1) \\
&= \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha. \mathit{deref}_{a_1 :_1 \alpha = f\ x}(f\ a_1) \\
&= \lambda_\omega f : \alpha \to_1 \alpha. \lambda_1 x : \alpha. (f\ (f\ x))
\end{aligned}$$

The main reason why this happens is because Linear Haskell uses *let* statements and, when applying the Launchbury semantics, most terms will generate lets. When we apply function deref_{Γ} to a let, it will save the variables in the environment, therefore we have to necessarily apply it to both abstractions and pairs, otherwise we will be left with free variables on a term.

The **substitution lemma** proves, by induction on the shape of the derivation tree, that applying the translating a $\mathcal{W}$-term where a substitution occurs to an $\mathcal{L}^{*}$-term is equal whether the operations are applied to the whole term or individually.

Lemma 7 (Substitution Lemma).
Given $M_{\mathcal{W}}[y \mapsto t]$*, then* $(M_{\mathcal{W}}[y \mapsto t])^{*}_{\mathcal{L}} = (M_{\mathcal{W}})^{*}_{\mathcal{L}}[y \mapsto t^{*}_{\mathcal{L}}]$.

Lemma 8 proves, by structural induction on the terms, that given an environment Γ and $\mathcal{L}$-term M, applying the deref_S function to Γ and M is equal whether we only translate to the $\mathcal{W}$-calculus or if we apply the Launchbury semantics and then translate.

Lemma 8. *Given* $(\Gamma; M_{\mathcal{L}})$*, then* $\mathit{deref}_{\Gamma_{\mathcal{W}}}((M_{\mathcal{L}}{}^{*})_{\mathcal{W}}) = \mathit{deref}_{\Gamma_{\mathcal{W}}}((M_{\mathcal{L}})_{\mathcal{W}})$.

Lemma 9 proves, by induction on the shape of the derivation tree, that if there exists a store with a variable with an $\mathcal{L}$-term t, where $(t^{*})_{\mathcal{W}}$ then we change that variable's value to $t_{\mathcal{W}}$ and, by applying the deref_S function, both computations yield the same result.

Lemma 9.
If $(S, a \mapsto q\ (t^*)_{\mathcal{W}}; M) \twoheadrightarrow (S_1; N_1)$ *then*

$$(S, a \mapsto q\ t_{\mathcal{W}}; M) \twoheadrightarrow (S_2; N_2) \text{ and } deref_{S_2}(N_2) = deref_{S_1}(N_1)$$

Finally, we proceed to the main results of our paper. On **theorem 1**, we prove by induction on the length of the derivation tree, that if we make a computation in the David Walker calculus then, if we translate the initial pair to Linear Haskell and reduce it, the $deref_{\Gamma}$ yields the same result in both computations.

By proving this theorem, we are able to show that we can simulate David Walker calculus in Linear Haskell. In **theorem 2**, we will prove the opposite simulation, also by induction on the length of the derivation tree, where we are able to show that if we make a computation in Linear Haskell then, if we translate the initial pair to the David Walker calculus and reduce it, the $deref_S$ yields the same result in both computations.

Theorem 1.
If $(S; M_{\mathcal{W}}) \rightarrow_k (S'; N_{\mathcal{W}})$ *then*

$$(S^*_{\mathcal{L}}; (M_{\mathcal{W}})^*_{\mathcal{L}}) \Downarrow (\Gamma; N_{\mathcal{L}}) \text{ and } deref_{\Gamma}(N_{\mathcal{L}}) = deref_{S'^*_{\mathcal{L}}}((N_{\mathcal{W}})^*_{\mathcal{L}})$$

Theorem 2.
If $(\Gamma; M^*_{\mathcal{L}}) \Downarrow (\Gamma'; N_{\mathcal{L}})$ *then*

$$(\Gamma_{\mathcal{W}}; (M_{\mathcal{L}})_{\mathcal{W}}) \twoheadrightarrow (S; N_{\mathcal{W}}) \text{ and } deref_{\mathcal{S}}(N_{\mathcal{W}}) = deref_{\Gamma'_{\mathcal{W}}}((N_{\mathcal{L}})_{\mathcal{W}})$$

These two main theorems show that, in fact, there is an equivalence between these two systems, in the sense that it is possible to define compilations between them which preserve reduction.

5 Conclusions

In this paper, we defined two compilations that relate two different linear calculi, the David Walker calculus [12], a linear type system which uses a small-step operational semantics, and Linear Haskell [4], an extension to Haskell, that uses a big-step semantics. We were able to show that there exists an equivalence between both systems, despite several differences in syntax and the kind of operational semantics (small-step and big-step) used.

Acknowledgements This work was partially financially supported by the Artificial Intelligence and Computer Science Laboratory, LIACC, funded by national funds through the FCT/MCTES (PIDDAC) - UIDB/00027/2020.

References

1. Alves, S., Fernández, M., Florido, M., Mackie, I.: Linearity and recursion in a typed lambda-calculus. In: Proceedings of the 13th International ACM Conference on Principles and Practice of Declarative Programming (PPDP) (2011)

2. Alves, S., Fernández, M., Florido, M., Mackie, I.: Linearity: a roadmap. Journal of Logic and Computation **24**(3), 513–529 (2014)
3. Barendregt, H.P.: The lambda calculus - its syntax and semantics, Studies in logic and the foundations of mathematics, vol. 103. North-Holland (1985)
4. Bernardy, J.P., Boespflug, M., Newton, R.R., Peyton Jones, S., Spiwack, A.: Linear haskell: practical linearity in a higher-order polymorphic language. Proc. of the 44th ACM Symposium on Principles of Programming Languages **2**(POPL) (2017)
5. Caires, L., Pfenning, F., Toninho, B.: Linear logic propositions as session types. Mathematical Structures in Computer Science **26**(3), 367–423 (2016)
6. Coppola, P., Lago, U.D., Rocca, S.R.D.: Elementary affine logic and the call-by-value lambda calculus. In: Typed Lambda Calculi and Applications, (TLCA). Lecture Notes in Computer Science, vol. 3461, pp. 131–145. Springer (2005)
7. Girard, J.Y.: Linear logic. Theoretical Computer Science **50**(1), 1–101 (1987)
8. Hughes, J., Orchard, D.: Program synthesis from graded types. In: Proceedings of the 33rd European Symposium on Programming (ESOP). Lecture Notes in Computer Science, vol. 14576, pp. 83–112. Springer (2024)
9. Launchbury, J.: A natural semantics for lazy evaluation. In: Proceedings of the 20th ACM Symposium on Principles of Programming Languages (POPL) (1993)
10. Mackie, I.: Lilac: A functional programming language based on linear logic. Journal of Functional Programming **4**(4), 395–433 (1994)
11. Maraist, J., Odersky, M., Turner, D.N., Wadler, P.: Call-by-name, call-by-value, call-by-need and the linear lambda calculus. Theoretical Computer Science **228**(1-2), 175–210 (1999)
12. Walker, D.: Substructural type systems. In: Pierce, B.C. (ed.) Advanced Topics in Types and Programming Languages. pp. 3–44. The MIT Press (2005)

6 Appendix (Proofs)

Lemma 1.

1. *Given* $(S; t_1\ t_2) \rightarrow_k (S''; v)$*, then* $\exists k_1, k_2 \in \mathbb{N}$*,* $k = k_1 + k_2$ *such that*

$$(S; t_1\ t_2) \rightarrow_{k_1} (S'; x\ t_2) \rightarrow_{k_2} (S''; v)$$

Proof By induction on k:
- k = 0 holds vacuously because $(S; t_1\ t_2)$ and $(S''; v)$ are different.
- Assume the lemma holds for $k \leq k_0$ and we shall prove it for $k_0 + 1$.
  Assume that $(S; t_1\ t_2) \rightarrow_{k_0+1} (S''; v)$.
  This means that the derivation sequence can be written as

$$(S; t_1\ t_2) \rightarrow \gamma \rightarrow_{k_0} (S''; v)$$

for some configuration Γ.
If we apply the system rule $(S; E[t]) \rightarrow (S'; E[t'])$, we have $\gamma = (S_1; t'_1\ t_2)$ and

$$(S; t_1\ t_2) \rightarrow (S_1; t'_1\ t_2)$$

because

$$(S; t_1) \rightarrow (S_1; t'_1)$$

We therefore have

$$(S_1; t'_1\ t_2) \rightarrow_{k_0} (S''; v)$$

which is true by induction hypothesis.
This means that there are natural numbers k_1 and k_2, such that

$$(S_1; t'_1) \rightarrow_{k_1} (S'; x) \text{ and } (S'; x\ t_2) \rightarrow_{k_2} (S'', v)$$

,
where $k_0 = k_1 + k_2$.
Using the fact that $(S; t_1) \rightarrow (S_1; t'_1)$ and $(S_1; t'_1) \rightarrow_{k_1} (S'; x)$, we get

$$(S; t_1) \rightarrow_{k_1+1} (S'; x)$$

We have already seen that $(S'; x\ t_2) \rightarrow_{k_2} (S''; v)$ and, since $(k_1+1)+k_2 = k_0 + 1$, we have proven the required result.

2. *Given* $(S; \mathit{split}\ t_1\ \mathit{as}\ y, z\ \mathit{in}\ t_2) \rightarrow_k (S''; v)$*, then* $\exists k_1, k_2 \in \mathbb{N}, k = k_1 + k_2$ *such that*

$$(S; \mathit{split}\ t_1\ \mathit{as}\ y, z\ \mathit{in}\ t_2) \rightarrow_{k_1} (S'; \mathit{split}\ x\ \mathit{as}\ y, z\ \mathit{in}\ t_2) \rightarrow_{k_2} (S''; v)$$

Proof By induction on k:

- k = 0 holds vacuously because $(S; \text{split } t_1 \text{ as } y, z \text{ in } t_2)$ and $(S''; v)$ are different.
- Assume the lemma holds for $k \leq k_0$ and we shall prove it for $k_0 + 1$.
  Assume that $(S; \text{split } t_1 \text{ as } y, z \text{ in } t_2) \rightarrow_{k_0+1} (S''; v)$.
  This means that the derivation sequence can be written as

  $$(S; \text{split } t_1 \text{ as } y, z \text{ in } t_2) \rightarrow \gamma \rightarrow_{k_0} (S''; v)$$

  for some configuration Γ.
  If we apply the system rule $(S; E[t]) \rightarrow (S'; E[t'])$, we have

  $$\gamma = (S_1; \text{split } t'_1 \text{ as } y, z \text{ in } t_2)$$

  and

  $$(S; \text{split } t_1 \text{ as } y, z \text{ in } t_2) \rightarrow (S_1; \text{split } t'_1 \text{ as } y, z \text{ in } t_2)$$

  because

  $$(S; t_1) \rightarrow (S_1; t'_1)$$

  We therefore have

  $$(S_1; \text{split } t'_1 \text{ as } y, z \text{ in } t_2) \rightarrow_{k_0} (S''; v)$$

  which is true by induction hypothesis.
  This means that there are natural numbers k_1 and k_2, such that

  $$(S_1; t'_1) \rightarrow_{k_1} (S'; x)$$

  and

  $$(S'; \text{split } x \text{ as } y, z \text{ in } t_2) \rightarrow_{k_2} (S'', v)$$

  where $k_0 = k_1 + k_2$. Using the fact that

  $$(S; t_1) \rightarrow (S_1; t'_1) \text{ and } (S_1; t'_1) \rightarrow_{k_1} (S'; x)$$

  we get

  $$(S; t_1) \rightarrow_{k_1+1} (S'; x)$$

  We have already seen that $(S'; \text{split } x \text{ as } y, z \text{ in } t_2) \rightarrow_{k_2} (S''; v)$ and, since $(k_1 + 1) + k_2 = k_0 + 1$, we have proven the required result.

Lemma 2 ($deref_S$ lemma).
Given $S, a \mapsto q\ e$, then $deref_{S,a \mapsto q\ e}(t) = deref_S(t[a \mapsto e])$.

<u>Proof</u> By induction on t:

– $t \equiv a$
We have

$$\begin{aligned} & deref_{S,a=q\ e}(a) \\ = & deref_S(a[a \mapsto e]) \\ & \text{by proposition 1, we can remove } a \text{ from the heap} \\ = & deref_S(e) \end{aligned}$$

which is true by definition.

– $t \equiv q_1\ \lambda y : q_2\ P.\ t_1$
We have

$$\begin{aligned} & deref_{S,a=q\ e}(q_1\ \lambda y : q_2\ P.\ t_1) \\ = & \text{let } (v, S') = deref_{S,a=q\ e}(t_1) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_S(t_1[a \mapsto e]) \\ & \text{in } (q_1\ \lambda y : q_2\ P.\ v, S') \end{aligned}$$

So the property holds.

– $t \equiv e_1\ e_2$
There are two cases:

- q = un
  We have

$$\begin{aligned} & deref_{S,a=\text{ un } e}(e_1\ e_2) \\ = & \text{let } (v_1, S_1) = deref_{S,a=\text{ un } e}(e_1) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_S(e_1[a \mapsto e]) \\ & \quad (v_2, S_2) = deref_{S_1,a=\text{ un } e}(e_2) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_{S_1}(e_2[a \mapsto e]) \\ & \text{in } (v_1\ v_2, S_2) \end{aligned}$$

- q = lin
  If $x \in fv(e_1)$ we have

$$\begin{aligned} & deref_{S,a=\text{ lin } e}(e_1\ e_2) \\ = & \text{let } (v_1, S_1) = deref_{S,a=\text{ lin } e}(e_1) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_S(e_1[a \mapsto e]) \\ & \quad (v_2, S_2) = deref_{S_1}(e_2) \\ & \text{in } (v_1\ v_2, S_2) \end{aligned}$$

If $x \in fv(e_2)$ we have

$$\begin{aligned}
& deref_{S,a=\text{ lin }e}(e_1\ e_2) \\
= & \text{ let } (v_1, S_1) = deref_S(e_1) \\
& \quad (v_2, S_2) = deref_{S_1,a=\text{ lin }e}(e_2) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_{S_1}(e_2[a \mapsto e]) \\
& \text{in } (v_1\ v_2, S_2)
\end{aligned}$$

– $t \equiv q_1\ < e_1, e_2 >$
There are two cases:

- q = un
  We have

$$\begin{aligned}
& deref_{S,a=\text{ un }e}(q_1\ < e_1, e_2 >) \\
= & \text{ let } (v_1, S_1) = deref_{S,a=\text{ un }e}(e_1) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_S(e_1[a \mapsto e]) \\
& \quad (v_2, S_2) = deref_{S_1,a=\text{ un }e}(e_2) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_{S_1}(e_2[a \mapsto e]) \\
& \text{in } (q_1\ < v_1, v_2 >, S_2)
\end{aligned}$$

- q = lin
  If $x \in fv(e_1)$ we have

$$\begin{aligned}
& deref_{S,a=\text{ lin }e}(q_1\ < e_1, e_2 >) \\
= & \text{ let } (v_1, S_1) = deref_{S,a=\text{ lin }e}(e_1) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_S(e_1[a \mapsto e]) \\
& \quad (v_2, S_2) = deref_{S_1}(e_2) \\
& \text{in } (q_1\ < v_1, v_2 >, S_2)
\end{aligned}$$

  If $x \in fv(e_2)$ we have

$$\begin{aligned}
& deref_{S,a=\text{ lin }e}(q_1\ < e_1, e_2 >) \\
= & \text{ let } (v_1, S_1) = deref_S(e_1) \\
& \quad (v_2, S_2) = deref_{S_1,a=\text{ lin }e}(e_2) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_{S_1}(e_2[a \mapsto e]) \\
& \text{in } (q_1\ < v_1, v_2 >, S_2)
\end{aligned}$$

– $t \equiv \text{split } e_1 \text{ as } y, z \text{ in } e_2$
  There are two cases:
  - q = un
    We have

$$\begin{aligned} & deref_{S,a=\text{ un } e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\ = & \text{ let } (v_1, S_1) = deref_{S,a=\text{ un } e}(e_1) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_S(e_1[a \mapsto e]) \\ & (v_2, S_2) = deref_{S_1,a=\text{ un } e}(e_2) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_{S_1}(e_2[a \mapsto e]) \\ & \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, S_2) \end{aligned}$$

  - q = lin
    If $x \in fv(e_1)$ we have

$$\begin{aligned} & deref_{S,a=\text{ lin } e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\ = & \text{ let } (v_1, S_1) = deref_{S,a=\text{ lin } e}(e_1) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_S(e_1[a \mapsto e]) \\ & (v_2, S_2) = deref_{S_1}(e_2) \\ & \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, S_2) \end{aligned}$$

    If $x \in fv(e_2)$ we have

$$\begin{aligned} & deref_{S,a=\text{ lin } e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\ = & \text{ let } (v_1, S_1) = deref_S(e_1) \\ & (v_2, S_2) = deref_{S_1,a=\text{ lin } e}(e_2) \\ & \qquad \text{by induction hypothesis,} \\ & \qquad = deref_{S_1}(e_2[a \mapsto e]) \\ & \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, S_2) \end{aligned}$$

Lemma 3 (S Substitution Lemma).
If $(S, a \mapsto q\ t; M) \twoheadrightarrow (S_1; N_1)$ *then*

$$(S; M[a \mapsto t]) \twoheadrightarrow (S_2; N_2) \text{ and } deref_{S_2}(N_2) = deref_{S_1}(N_1)$$

<u>Proof</u> By induction on $(S; t) \twoheadrightarrow (S'; v)$:

– Given $(S, a \mapsto q\ e; q_1\ \lambda y : q_2\ P.\ t)$, we have, by rule *s-abs*$_{\mathcal{W}}$,

$$\begin{aligned} & (S, a \mapsto q\ e; q_1\ \lambda y : q_2\ P.\ t) \\ \to & (S, a \mapsto q\ e, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t; a_1) \end{aligned}$$

We know that, since $q_1\ \lambda y : q_2\ P.\ \mathrm{t}$ is not evaluated, $a \notin fv(q_1\ \lambda y : q_2\ P.\ t)$, so we have $(q_1\ \lambda y : q_2\ P.\ t)[a \mapsto e] = q_1\ \lambda y : q_2\ P.\ \mathrm{t}$, for which, by rule s-abs$_{\mathcal{W}}$,

$$\begin{aligned}&(S; q_1\ \lambda y : q_2\ P.\ t)\\ \to\ &(S, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t; a_2)\end{aligned}$$

We know that

$$\begin{aligned}&dereff_{S,a\mapsto q\ e,a_1\mapsto q_1\ \lambda y:q_2\ P.\ t}(a_1)\\ =\ &dereff_{S,a\mapsto q\ e}(q_1\ \lambda y : q_2\ P.\ t)\\ &\text{since } a_1 \text{ is a fresh variable, it can be removed from the heap}\\ =\ &\text{let } (v, S') = dereff_{S,a\mapsto q\ e}(t)\\ &\qquad\qquad \text{since we know that } a \notin fv(t)\\ &\qquad\qquad = dereff_S(t), \text{ by the } dereff_S \text{ lemma}\\ &\text{in } (q_1\ \lambda y : q_2\ P.\ v, S')\end{aligned}$$

and

$$\begin{aligned}&dereff_{S,a_2\mapsto q_1\ \lambda y:q_2\ P.\ t}(a_2)\\ =\ &dereff_S(q_1\ \lambda y : q_2\ P.\ t)\\ &\text{since } a_2 \text{ is a fresh variable, it can be removed from the heap}\end{aligned}$$

We already know that $(v, S') = dereff_S(t)$, therefore

$$dereff_S(q_1\ \lambda y : q_2\ P.\ t) = (q_1\ \lambda y : q_2\ P.\ v, S')$$

So the property holds.

– Given $(S, a \mapsto q\ e; q_1 < t_1, t_2 >)$, we have

$$\frac{}{(S, a \mapsto q\ e; q_1 < t_1, t_2 >) \to (S, a \mapsto q\ e, a_1 \mapsto q_1 < t_1, t_2 >; a_1)}\ \textit{s-pair}_{\mathcal{W}}$$

We know that, since $q_1 < t_1, t_2 >$ is not evaluated, $a \notin fv(q_1 < t_1, t_2 >)$, we have

$$\begin{aligned}(q_1 < t_1, t_2 >)[a \mapsto e] &= q_1 < t_1[a \mapsto e], t_2[a \mapsto e] >\\ &= q_1 < t_1, t_2 >\end{aligned}$$

and

$$\frac{}{(S; q_1 < t_1, t_2 >) \to (S, a_2 \mapsto q_1 < t_1, t_2 >; a_2)}\ \textit{s-pair}_{\mathcal{W}}$$

Since we know that $a \notin q_1 < t_1, t_2 >$, we do not need to prove this for each possibility of q, so we will then assume $q = \text{un}$. We know that,

$$
\begin{aligned}
& deref_{S,a\mapsto q\ e,a_1\mapsto q_1<t_1,t_2>}(a_1) \\
= & \ deref_{S,a\mapsto q\ e}(q_1 < t_1, t_2 >) \\
& \text{since } a_1 \text{ is a fresh variable, it can be removed from the heap} \\
= & \ \text{let } (v_1, S_1) = deref_{S,a\mapsto q\ e}(t_1) \\
& \qquad \text{since we know that } a \notin fv(t_1) \\
& \qquad = deref_S(t_1), \text{ by the } deref_S \text{ lemma} \\
& \quad (v_2, S_2) = deref_{S_1,a\mapsto q\ e}(t_2) \\
& \qquad \text{since we know that } a \notin fv(t_2) \\
& \qquad = deref_{S_1}(t_2), \text{ by the } deref_S \text{ lemma} \\
& \text{in } (q_1 < v_1, v_2 >, S_2)
\end{aligned}
$$

and

$$
\begin{aligned}
& deref_{S,a_2\mapsto q_1 q_1<t_1,t_2>}(a_2) \\
= & \ deref_S(q_1 < t_1, t_2 >) \\
& \text{since } a_2 \text{ is a fresh variable, it can be removed from the heap}
\end{aligned}
$$

We know that $(v_1, S_1) = deref_S(t_1)$ and $(v_2, S_2) = deref_{S_1}(t_2)$, therefore

$$deref_S(q_1 < t_1, t_2 >) = (q_1 < v_1, v_2 >, S_2)$$

So the property holds.

– Given $(S, a \mapsto q\ e; x\ t)$, we have

$$\frac{S(x) = q_1\ \lambda y : q_2\ P.\ t'}{(S, a \mapsto q\ e; x\ t) \to (S \sim^q x, a \mapsto q\ e; t'[y \mapsto t])}\ \textit{s-app}_{\mathcal{W}}$$

We know that if x occurs unrestricted then it will stay in S but, if x occurs linearly then it will not be used again, thus we can use $S \sim^q x = S$.
So there are two cases,

- $x = a$
  Since we know that $(x\ t)[a \mapsto e] = x[a \mapsto e]\ t[a \mapsto e] = e\ t[a \mapsto e]$, then $e \equiv q_1\ \lambda y : q_2\ P.\ t'$ because its value must be equal to $S(x)$, so we have

  $$(S; (q_1\ \lambda y : q_2\ P.\ t')\ t[a \mapsto e]) \to (S, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_1\ t[a \mapsto e])$$

  and, by rule $\textit{s-app}_{\mathcal{W}}$

  $$(S, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_1\ t[a \mapsto e]) \to (S \sim^{q_1} a_1; t'[y \mapsto t[a \mapsto e]])$$

  We know that a_1 is a fresh variable, so, since it will not be used again, we can use $S \sim^{q_1} a_1 = S$.

Now we have

$$\begin{aligned} & deref_{S,a\mapsto q\ e}(t'[y \mapsto t]) \\ = & deref_S(t'[y \mapsto t][a \mapsto e]) \text{ by the } deref_S \text{ lemma} \end{aligned}$$

We know that, since t' is not evaluated and a is an address variable, then $a \notin fv(t')$, therefore we can conclude that

$$deref_S(t'[y \mapsto t][a \mapsto e]) = deref_S(t'[y \mapsto t[a \mapsto e]])$$

We know that $(S, a \mapsto q\ e; t'[y \mapsto t]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t')$,

$$(S; t'[y \mapsto t[a \mapsto e]]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

- $x \neq a$
  Since we know that $(x\ t)[a \mapsto e] = x[a \mapsto e]\ t[a \mapsto e] = x\ t[a \mapsto e]$, so we have

$$\frac{S(x) = q_1\ \lambda y : q_2\ P.\ t'}{(S; x\ t[a \mapsto e]) \to (S \sim^{q_1} x; t'[y \mapsto t[a \mapsto e]])}\ \textit{s-app}_{\mathcal{W}}$$

  We know that if x occurs unrestricted then it will stay in S but, if it occurs linearly then it will not be used again, thus we can use $S \sim^{q_1} x = S$. Then we have

$$\begin{aligned} & deref_{S,a\mapsto q\ e}(t'[y \mapsto t]) \\ = & deref_S(t'[y \mapsto t][a \mapsto e]) \text{ by the } deref_S \text{ lemma} \end{aligned}$$

  We know that, since t' is not evaluated, and both a and x are address variables, $a, x \notin fv(t')$, therefore we can conclude that

$$deref_S(t'[y \mapsto t][a \mapsto e]) = deref_S(t'[y \mapsto t[a \mapsto e]])$$

  We know that $(S, a \mapsto q\ e; t'[y \mapsto t]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t')$,

$$(S; t'[y \mapsto t[a \mapsto e]]) \twoheadrightarrow (S_2; v_2)$$

  and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

– Given $(S, a \mapsto q\ e; (q_1\ \lambda y : q_2\ P.\ t_1)\ t_2)$, we have

$$(S, a \mapsto q\ e; (q_1\ \lambda y : q_2\ P.\ t_1)\ t_2) \to (S, a \mapsto q\ e, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t_1; a_1\ t_2)$$

  and, by rule $\textit{s-app}_{\mathcal{W}}$

$$\begin{aligned} & (S, a \mapsto q\ e, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t_1; a_1\ t_2) \\ \to & (S \sim^{q_1} a_1, a \mapsto q\ e; t_1[y \mapsto t_2]) \end{aligned}$$

We know that $q_1\ \lambda y : q_2\ P.\ t_1$ is not evaluated and S saves address variables contained in t_2. Therefore we know that $a \notin fv(t_1)$, so

$$((q_1\ \lambda y : q_2\ P.\ t_1)\ t_2)[a \mapsto e] = (q_1\ \lambda y : q_2\ P.\ t_1)\ t_2[a \mapsto e]$$

Then, we have

$$(S; (q_1\ \lambda y : q_2\ P.\ t_1)\ t_2[a \mapsto e]) \to (S, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t_1; a_2\ t_2[a \mapsto e])$$

and, by rule *s-app*$_{\mathcal{W}}$

$$\begin{aligned} &(S, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t_1; a_2\ t_2[a \mapsto e]) \\ \to\ &(S \sim^{q_1} a_2; t_1[y \mapsto t_2[a \mapsto e]]) \end{aligned}$$

Since both a_1 and a_2 are fresh variables, they will not be used again, so we can use $S \sim^{q_1} a_1 = S$ and $S \sim^{q_1} a_2 = S$. Thus

$$\begin{aligned} deref_{S, a \mapsto q\ e}(t_1[y \mapsto t_2]) &= deref_S(t_1[y \mapsto t_2][a \mapsto e]) \text{ by the } deref_S \text{ lemma} \\ &\quad \text{since } a \notin fv(t_1), \text{ we have} \\ &= deref_S(t_1[y \mapsto t_2[a \mapsto e]]) \end{aligned}$$

We know that $(S, a \mapsto q\ e; t_1[y \mapsto t_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t_1)$,

$$(S; t_1[y \mapsto t_2[a \mapsto e]]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

– Given $(S, a \mapsto q\ e; \text{split } x \text{ as } y, z \text{ in } t_2)$, we have, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned} &(S, a \mapsto q\ e; \text{split } x \text{ as } y, z \text{ in } t) \\ \to\ &(S \sim^{q_1} x, a \mapsto q\ e; t[y \mapsto t_1][z \mapsto t_2]) \end{aligned}$$

where $S(x) = q_1 < t_1, t_2 >$
We know that if x occurs unrestricted then it will stay in S but, if it occurs linearly then it will not be used again, thus we can use $S \sim^{q_1} x = S$.
So there are two cases,

- $x = a$
  Since we know that

  $$\begin{aligned} (\text{split } x \text{ as } y, z \text{ in } t)[a \mapsto e] &= \text{split } x[a \mapsto e] \text{ as } y, z \text{ in } t[a \mapsto e] \\ &= \text{split } e \text{ as } y, z \text{ in } t[a \mapsto e] \end{aligned}$$

  then $e \equiv q_1 < t_1, t_2 >$ because its value must be equal to $S(x)$, so we have

  $$\begin{aligned} &(S; \text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t[a \mapsto e]) \\ \to\ &(S, a_1 \mapsto q_1 < t_1, t_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t[a \mapsto e]) \end{aligned}$$

and, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned}&(S, a_1 \mapsto q_1 < t_1, t_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t[a \mapsto e])\\ \to\ &(S \sim^{q_1} a_1; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])\end{aligned}$$

We know that a_1 is a fresh variable, so it will not be used anymore and we can use $S \sim^{q_1} a_1 = S$.
Now we have

$$\begin{aligned}&deref_{S,a\mapsto q\ e}(t[y \mapsto t_1][z \mapsto t_2])\\ =\ &deref_S(t[y \mapsto t_1][z \mapsto t_2][a \mapsto e]) \text{ by the } deref_S \text{ lemma}\end{aligned}$$

We know that, since t_1 and t_2 are not evaluated and a is an address variable, then $a \notin fv(t_1)$ and $a \notin fv(t_2)$, therefore we can conclude that

$$deref_S(t[y \mapsto t_1][z \mapsto t_2][a \mapsto e]) = deref_S(t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])$$

We know $(S, a \mapsto q\ e; t[y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t_1)$ and $a \notin fv(t_2)$, $(S; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_2; v_2)$ and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

- $x \neq a$
  Since we know that

$$\begin{aligned}&(\text{split } x \text{ as } y, z \text{ in } t)[a \mapsto e]\\ =\ &\text{split } x[a \mapsto e] \text{ as } y, z \text{ in } t[a \mapsto e]\\ =\ &\text{split } x \text{ as } y, z \text{ in } t[a \mapsto e]\end{aligned}$$

  we have, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned}&(S; \text{split } x \text{ as } y, z \text{ in } t[a \mapsto e])\\ \to\ &(S \sim^{q_1}; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])\end{aligned}$$

  where $S(x) = q_1 < t_1, t_2 >$.
  We know that if x occurs unrestricted then it will stay in S but, if it occurs linearly then it will not be used again, thus we can use
  $S \sim^{q_1} x = S$ Then we have

$$deref_{S,a\mapsto q\ e}(t[y \mapsto t_1][z \mapsto t_2]) = deref_S(t[y \mapsto t_1][z \mapsto t_2][a \mapsto e])$$

  We know that, since t_1 and t_2 are not evaluated, and both a and x are address variables, then $a, x \notin fv(t_1)$ and $a, x \notin fv(t_2)$, therefore we can conclude that

$$deref_S(t[y \mapsto t_1][z \mapsto t_2][a \mapsto e]) = deref_S(t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])$$

  Since $(S, a \mapsto q\ e; t[y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t_1)$ and $a \notin fv(t_2)$, $(S; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_2; v_2)$ and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

– Given $(S, a \mapsto q\ e; \text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t)$, we have

$$\begin{aligned}&(S, a \mapsto q\ e; \text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t)\\ \rightarrow\ &(S, a \mapsto q\ e, a_1 \mapsto q_1 < t_1, t_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t)\end{aligned}$$

and, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned}&(S, a \mapsto q\ e, a_1 \mapsto q_1 < t_1, t_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t)\\ \rightarrow\ &(S \sim^{q_1} a_1, a \mapsto q\ e; t[y \mapsto t_1][z \mapsto t_2])\end{aligned}$$

We know that $q_1 < t_1, t_2 >$ is not evaluated and S saves address variables contained in t. Therefore we know that $a \notin fv(t_1)$ and $a \notin fv(t_2)$, so

$$\begin{aligned}&(\text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t)[a \mapsto e]\\ =\ &\text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t[a \mapsto e]\end{aligned}$$

Thus

$$\begin{aligned}&(S; \text{split } (q_1 < t_1, t_2 >) \text{ as } y, z \text{ in } t[a \mapsto e])\\ \rightarrow\ &(S, a_2 \mapsto q_1 < t_1, t_2 >; \text{split } a_2 \text{ as } y, z \text{ in } t[a \mapsto e])\end{aligned}$$

and, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned}&(S, a_2 \mapsto q_1 < t_1, t_2 >; \text{split } a_2 \text{ as } y, z \text{ in } t[a \mapsto e])\\ \rightarrow\ &(S \sim^{q_1} a_2; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])\end{aligned}$$

Since both a_1 and a_2 are fresh variables, they will not be used again, so we can use $S \sim^{q_1} a_1 = S$ and $S \sim^{q_1} a_2 = S$. Thus

$$\begin{aligned}&deref_{S, a \mapsto q\ e}(t[y \mapsto t_1][z \mapsto t_2])\\ =\ &deref_S(t[y \mapsto t_1][z \mapsto t_2][a \mapsto e]) \text{ by the } deref_S \text{ lemma}\\ &\text{since } a \notin fv(t_1) \text{ and } a \notin fv(t_2), \text{ we have}\\ =\ &deref_S(t[a \mapsto e][y \mapsto t_1][z \mapsto t_2])\end{aligned}$$

Since $(S, a \mapsto q\ e; t[y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, since we already know that $a \notin fv(t_1)$ and $a \notin fv(t_2)$,

$$(S; t[a \mapsto e][y \mapsto t_1][z \mapsto t_2]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. So the property holds.

Lemma 4 ($deref_\Gamma$ lemma).
Given $\Gamma, a :_\pi A = e$, then $deref_{\Gamma, a :_\pi A = e}(t) = deref_\Gamma(t[a \mapsto e])$.

<u>Proof</u> By induction on t:

– $t \equiv a$
We have

$$\begin{aligned}
& deref_{\Gamma, a:_\pi A=e}(a) \\
= & \; deref_\Gamma(a[a \mapsto e]) \\
& \text{by proposition 4, we can remove } x \text{ from the heap} \\
= & \; deref_\Gamma(e)
\end{aligned}$$

which is true by definition.
– $t \equiv \lambda_\pi y : P.\ t_1$
We have

$$\begin{aligned}
& deref_{\Gamma, a:_\pi A=e}(\lambda_\pi y : P.\ t_1) \\
= & \text{ let } (v, \Gamma') = deref_{\Gamma, a:_\pi A=e}(t_1) \\
& \qquad \text{by induction hypothesis,} \\
& \qquad = deref_\Gamma(t_1[a \mapsto e]) \\
& \text{in } (\lambda_\pi y : P.\ v, \Gamma')
\end{aligned}$$

So the property holds.
– $t \equiv e_1\ e_2$
There are two cases:
- $\pi = \omega$
  We have

$$\begin{aligned}
& deref_{\Gamma, a:_\omega A=e}(e_1\ e_2) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma, a:_\omega A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_\Gamma(e_1[a \mapsto e]) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1, a:_\omega A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (v_1\ v_2, \Gamma_2)
\end{aligned}$$

- $\pi = 1$
  If $x \in fv(e_1)$ we have

$$\begin{aligned}
& deref_{\Gamma, a:_1 A=e}(e_1\ e_2) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma, a:_1 A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_\Gamma(e_1[a \mapsto e]) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1}(e_2) \\
& \text{in } (v_1\ v_2, \Gamma_2)
\end{aligned}$$

If $x \in fv(e_2)$ we have

$$\begin{aligned}
& deref_{\Gamma,a:_1 A=e}(e_1\ e_2) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma}(e_1) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1,a:_1 A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (v_1\ v_2, \Gamma_2)
\end{aligned}$$

– $t \equiv < e_1, e_2 >_{\pi_1}$
There are two cases:

- $\pi = \omega$
  We have

$$\begin{aligned}
& deref_{\Gamma,a:_\omega A=e}(< e_1, e_2 >_{\pi_1}) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma,a:_\omega A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma}(e_1[a \mapsto e]) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1,a:_\omega A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (< v_1, v_2 >_{\pi_1}, \Gamma_2)
\end{aligned}$$

- $\pi = 1$
  If $x \in fv(e_1)$ we have

$$\begin{aligned}
& deref_{\Gamma,a:_1 A=e}(< e_1, e_2 >_{\pi_1}) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma,a:_1 A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma}(e_1[a \mapsto e]) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1}(e_2) \\
& \text{in } (< v_1, v_2 >_{\pi_1}, \Gamma_2)
\end{aligned}$$

  If $x \in fv(e_2)$ we have

$$\begin{aligned}
& deref_{\Gamma,a:_1 A=e}(< e_1, e_2 >_{\pi_1}) \\
= & \text{ let } (v_1, \Gamma_1) = deref_{\Gamma}(e_1) \\
& \quad (v_2, \Gamma_2) = deref_{\Gamma_1,a:_1 A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = deref_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (< v_1, v_2 >_{\pi_1}, \Gamma_2)
\end{aligned}$$

– $t \equiv \text{split } e_1 \text{ as } y, z \text{ in } e_2$
  There are two cases:
  - $\pi = \omega$
    We have

$$\begin{aligned}
& \mathit{deref}_{\Gamma, a:_\omega A=e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\
= \ & \text{let } (v_1, \Gamma_1) = \mathit{deref}_{\Gamma, a:_\omega A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = \mathit{deref}_{\Gamma}(e_1[a \mapsto e]) \\
& (v_2, \Gamma_2) = \mathit{deref}_{\Gamma_1, a:_\omega A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = \mathit{deref}_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, \Gamma_2)
\end{aligned}$$

  - $\pi = 1$
    If $x \in fv(e_1)$ we have

$$\begin{aligned}
& \mathit{deref}_{\Gamma, a:_1 A=e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\
= \ & \text{let } (v_1, \Gamma_1) = \mathit{deref}_{\Gamma, a:_1 A=e}(e_1) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = \mathit{deref}_{\Gamma}(e_1[a \mapsto e]) \\
& (v_2, \Gamma_2) = \mathit{deref}_{\Gamma_1}(e_2) \\
& \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, \Gamma_2)
\end{aligned}$$

    If $x \in fv(e_2)$ we have

$$\begin{aligned}
& \mathit{deref}_{\Gamma, a:_1 A=e}(\text{split } e_1 \text{ as } y, z \text{ in } e_2) \\
= \ & \text{let } (v_1, \Gamma_1) = \mathit{deref}_{\Gamma}(e_1) \\
& (v_2, \Gamma_2) = \mathit{deref}_{\Gamma_1, a:_1 A=e}(e_2) \\
& \qquad \text{by induction hypothesis} \\
& \qquad = \mathit{deref}_{\Gamma_1}(e_2[a \mapsto e]) \\
& \text{in } (\text{split } v_1 \text{ as } y, z \text{ in } v_2, \Gamma_2)
\end{aligned}$$

– $t \equiv \text{let}_{\pi_1}\ a_1 : B_1 = e_1, \ldots, a_n : B_n = e_n \text{ in t}$
  We have

$$\begin{aligned}
& \mathit{deref}_{\Gamma, a:_\pi A=e}(\text{let}_{\pi_1}\ a_1 : B_1 = e_1, \ldots, a_n : B_n = e_n \text{ in } t) \\
= \ & \mathit{deref}_{\Gamma, a:_\pi A=e, a_1:_{\pi_1} B_1=e_1, \ldots, a_n:_{\pi_1} B_n=e_n}(t).
\end{aligned}$$

  By induction hypothesis,

$$\begin{aligned}
& \mathit{deref}_{\Gamma, a:_\pi A=e, a_1:_{\pi_1} B_1=e_1, \ldots, a_n:_{\pi_1} B_n=e_n}(t) \\
= \ & \mathit{deref}_{\Gamma, a_1:_\pi B_1=e_1, \ldots, a_n:_\pi B_n=e_n}(t[a \mapsto e])
\end{aligned}$$

  The property holds.

Lemma 5.
Given that $x \notin fv(t)$*, if* $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)$ *then*

$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)$ *and* $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$

Proof By induction on $(\Gamma; t) \Downarrow (\Delta; v)$:

– Given $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \lambda_\pi x_1 : P.\ t)$, we have, by rule *s-abs*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \lambda_\pi x_1 : P.\ t)\\ \Downarrow\ &(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \lambda_\pi x_1 : P.\ t)\end{aligned}$$

And, by rule *s-abs*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; \lambda_\pi x_1 : P.\ t)\\ \Downarrow\ &(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; \lambda_\pi x_1 : P.\ t)\end{aligned}$$

We now have

$$\begin{aligned}&deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_{\pi_2} A_2 = e[z \mapsto x]}(\lambda_\pi x_1 : P.\ t)\\ =\ &\text{let } (v, \Gamma') = deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_{\pi_2} A_2 = e[z \mapsto x]}(t)\\ &\qquad \text{we know } y \notin fv(t[y \mapsto e[z \mapsto x]]), \text{ so by the } deref_\Gamma \text{ lemma}\\ &\qquad = deref_{\Gamma, x:_{\pi_1} A_1 = t_2}(t[y \mapsto e[z \mapsto x]])\\ &\qquad \text{we know that } x \notin fv(\lambda x_1 : P.\ t), \text{ so by proposition 3}\\ &\qquad = deref_\Gamma(t[y \mapsto e[z \mapsto t_2]]\\ &\text{in } (\lambda_\pi x_1 : P.\ v, \Gamma')\end{aligned}$$

and

$$deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_{\pi_2} A_2 = e[z \mapsto t_2]}(\lambda_\pi x_1 : P.\ t)$$

Since we know that $x \notin fv(\lambda x_1 : P.\ t)$ and $(v, \Gamma') = deref_\Gamma(t[y \mapsto e[z \mapsto t_2]]$, then

$$deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_{\pi_2} A_2 = e[z \mapsto t_2]}(\lambda_\pi x_1 : P.\ t) = (\lambda_\pi x_1 : P.\ v, \Gamma')$$

And the property holds.

– Given $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; < y_1, z_1 >_\pi)$, we have, by rule *s-pair*$_{\mathcal{L}}$,

$$\begin{aligned}&(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; < y_1, z_1 >_\pi)\\ \Downarrow\ &(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; < y_1, z_1 >_\pi)\end{aligned}$$

And, by rule *s-pair*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; < y_1, z_1 >_\pi)\\ \Downarrow\ &(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; < y_1, z_1 >_\pi)\end{aligned}$$

Now, there are two cases:

- $\pi_2 = \omega$
  We have

$$
\begin{aligned}
& dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_\omega A_2 = e[z \mapsto x]}(< y_1, z_1 >_\pi) \\
= \text{let } (v_1, \Gamma_1) &= dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_\omega A_2 = e[z \mapsto x]}(y_1) \\
&= dereff_{\Gamma, x:_{\pi_1} A_1 = t_2}(y_1[y \mapsto e[z \mapsto x]]) \text{ by the } dereff_\Gamma \text{ lemma} \\
&\text{we know that } x \notin fv(< y_1, z_1 >_\pi, \text{ so by proposition 3,} \\
&= dereff_\Gamma(y_1[y \mapsto e[z \mapsto t_2]]) \\
(v_2, \Gamma_2) &= dereff_{\Gamma_1, x:_{\pi_1} A_1 = t_2, y:_\omega A_2 = e[z \mapsto x]}(z_1) \\
&= dereff_{\Gamma_1, x:_{\pi_1} A_1 = t_2}(z_1[y \mapsto e[z \mapsto x]]) \text{ by the } dereff_\Gamma \text{ lemma} \\
&\text{we know that } x \notin fv(< y_1, z_1 >_\pi, \text{ so by proposition 3,} \\
&= dereff_{\Gamma_1}(z_1[y \mapsto e[z \mapsto t_2]]) \\
\text{in } (< v_1, v_2 >_\pi, \Gamma_2)
\end{aligned}
$$

  and

$$dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_\omega A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi)$$

  Since we know that $x \notin fv(< y_1, z_1 >_\pi$ and

  $(v_1, \Gamma_1) = dereff_\Gamma(y_1[y \mapsto e[z \mapsto t_2]])$ and $(v_2, \Gamma_2) = dereff_{\Gamma_1}(z_1[y \mapsto e[z \mapsto t_2]])$

  then

$$dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_\omega A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi) = (< v_1, v_2 >_\pi, \Gamma_2)$$

  And the property holds.
- $\pi_2 = 1$
  If $y \in fv(y_1)$, then

$$
\begin{aligned}
& dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto x]}(< y_1, z_1 >_\pi) \\
= \text{let } (v_1, \Gamma_1) &= dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto x]}(y_1) \\
&= dereff_{\Gamma, x:_{\pi_1} A_1 = t_2}(y_1[y \mapsto e[z \mapsto x]]) \text{ by the } dereff_\Gamma \text{ lemma} \\
&\text{we know that } x \notin fv(< y_1, z_1 >_\pi, \text{ so by proposition 3,} \\
&= dereff_\Gamma(y_1[y \mapsto e[z \mapsto t_2]]) \\
(v_2, \Gamma_2) &= dereff_{\Gamma_1}(z_1) \\
\text{in } (< v_1, v_2 >_\pi, \Gamma_2)
\end{aligned}
$$

  and

$$dereff_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi)$$

  Since we know that $x \notin fv(< y_1, z_1 >_\pi$ and

$$(v_1, \Gamma_1) = dereff_\Gamma(y_1[y \mapsto e[z \mapsto t_2]]) \text{ and } (v_2, \Gamma_2) = dereff_{\Gamma_1}(z_1)$$

then

$$deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi) = (< v_1, v_2 >_\pi, \Gamma_2)$$

And the property holds.
If $y \in fv(z_1)$, then

$$\begin{aligned}
& deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto x]}(< y_1, z_1 >_\pi) \\
= \text{let } (v_1, \Gamma_1) &= deref_{\Gamma, x:_{\pi_1} A_1 = t_2}(y_1) \\
& \text{we know that } x \notin fv(< y_1, z_1 >_\pi, \text{ it can be removed} \\
& = deref_\Gamma(y_1) \\
(v_2, \Gamma_2) &= deref_{\Gamma_1, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto x]}(z_1) \\
& = deref_{\Gamma_1, x:_{\pi_1} A_1 = t_2}(z_1[y \mapsto e[z \mapsto x]]) \text{ by the } deref_\Gamma \text{ lemma} \\
& \text{we know that } x \notin fv(< y_1, z_1 >_\pi, \text{ so by proposition 3,} \\
& = deref_\Gamma(z_1[y \mapsto e[z \mapsto t_2]]) \\
\text{in } (< v_1, v_2 >_\pi, \Gamma_2) &
\end{aligned}$$

and

$$deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi)$$

Since we know that $x \notin fv(< y_1, z_1 >_\pi$ and $(v_1, \Gamma_1) = deref_\Gamma(y_1)$ and $(v_2, \Gamma_2) = deref_{\Gamma_1}(z_1[y \mapsto e[z \mapsto t_2]])$, then

$$deref_{\Gamma, x:_{\pi_1} A_1 = t_2, y:_1 A_2 = e[z \mapsto t_2]}(< y_1, z_1 >_\pi) = (< v_1, v_2 >_\pi, \Gamma_2)$$

And the property holds.

- Given $(\Gamma, y_1 :_\omega B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; y_1)$, we have

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)}{(\Gamma, y_1 :_\omega B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; y_1) \Downarrow (\Delta_1, y_1 :_\omega B = v_1; v_1)} \; \textit{s-unvar}_{\mathcal{L}}$$

and

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)}{(\Gamma, y_1 :_\omega B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; y_1) \Downarrow (\Delta_2, y_1 :_\omega B = v_2; v_2)} \; \textit{s-unvar}_{\mathcal{L}}$$

Since we know that $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)$ then, by induction hypothesis, there is

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$.
We also know that $y_1 \notin fv(v_1)$ and $y_2 \notin fv(v_2)$, so we can conclude that $deref_{\Delta_2, y_1:_\omega B = v_2}(v_2) = deref_{\Delta_1, y_1:_\omega B = v_1}(v_1)$.

- Given $(\Gamma, y_1 :_1 B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; y_1)$, we have

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)}{(\Gamma, y_1 :_1 B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; y) \Downarrow (\Delta_1; v_1)} \; \textit{s-linvar}_{\mathcal{L}}$$

and

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)}{(\Gamma, y_1 :_1 B = t, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; y) \Downarrow (\Delta_2; v_2)} \; s\text{-}linvar_{\mathcal{L}}$$

Since we know that $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; v_1)$ then, by induction hypothesis, there is

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$. So the property holds.

- Given $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t\ y_1)$, we have

$$\frac{\begin{array}{c}(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; \lambda_\pi z_1 : P.t') \\ (\Delta_1; t'[z_1 \mapsto y_1]) \Downarrow (\Theta_1; v_1)\end{array}}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t\ y_1) \Downarrow (\Theta_1; v_1)} \; s\text{-}app_{\mathcal{L}}$$

and

$$\frac{\begin{array}{c}(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; \lambda_\pi z_2 : P.t'') \\ (\Delta_2; t''[z_2 \mapsto y_1]) \Downarrow (\Theta_2; v_2)\end{array}}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t\ y_1) \Downarrow (\Theta_2; v_2)} \; s\text{-}app_{\mathcal{L}}$$

Since we know that

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; t) \Downarrow (\Delta_1; \lambda_\pi z_1 : P.t')$$

then, by induction hypothesis, there is

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; t) \Downarrow (\Delta_2; \lambda_\pi z_2 : P.t'')$$

and $deref_{\Delta_1}(\lambda_\pi z_1 : P.t') = deref_{\Delta_2}(\lambda_\pi z_2 : P.t'')$. From this, we know that $deref_{\Delta_1}(t') = deref_{\Delta_2}(t'')$ then, by α-conversion, we can conclude that $deref_{\Theta_1}(v_1) = deref_{\Theta_2}(v_2)$.

- Given $(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \text{split } e \text{ as } y_1, z_1 \text{ in } t)$, we have

$$\frac{\begin{array}{c}(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; e) \Downarrow (\Delta_1; < y_2, z_2 >_\pi) \\ (\Delta_1; t[y_1 \mapsto y_2][z_1 \mapsto z_2]) \Downarrow (\Theta_1; v_1)\end{array}}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \text{split } e \text{ as } y_1, z_1 \text{ in } t) \Downarrow (\Theta_1; v_1)} \; s\text{-}split_{\mathcal{L}}$$

and

$$\frac{\begin{array}{c}(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; e) \Downarrow (\Delta_2; < y_3, z_3 >_\pi) \\ (\Delta_2; t[y_1 \mapsto y_3][z_1 \mapsto z_3]) \Downarrow (\Theta_2; v_2)\end{array}}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; \text{split } e \text{ as } y_1, z_1 \text{ in } t) \Downarrow (\Theta_2; v_2)} \; s\text{-}split_{\mathcal{L}}$$

Since we know that

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; e) \Downarrow (\Delta_1; < y_2, z_2 >_\pi)$$

then, by induction hypothesis, there is

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; e) \Downarrow (\Delta_2; < y_3, z_3 >_\pi)$$

and $deref_{\Delta_1}(< y_2, z_2 >_\pi) = deref_{\Delta_2}(< y_3, z_3 >_\pi)$.
Since we know that

$$\begin{aligned} deref_{\Delta_2}(< y_3, z_3 >_\pi) = \text{let } & (v_1, \Delta_2{}') = deref_{\Delta_2}(y_3) \\ & (v_2, \Delta_2{}'') = deref_{\Delta_2{}'}(z_3) \\ & \text{in } (< v_1, v_2 >_\pi, \Delta_2{}'') \end{aligned}$$

and

$$\begin{aligned} deref_{\Delta_1}(< y_2, z_2 >_\pi) = \text{let } & (v_3, \Delta_1{}') = deref_{\Delta_1}(y_2) \\ & (v_4, \Delta_1{}'') = deref_{\Delta_1{}'}(z_2) \\ & \text{in } (< v_3, v_4 >_\pi, \Delta_1{}'') \end{aligned}$$

then

$$deref_{\Delta_1}(y_2) = deref_{\Delta_2}(y_3) \text{ and } deref_{\Delta_1{}'}(z_2) = deref_{\Delta_2{}'}(z_3)$$

So we can conclude that $deref_{\Theta_1}(v_1) = deref_{\Theta_2}(v_2)$.

- Given

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \text{let}_\pi\ a_1 : B_1 = e_1, \ldots, a_n : B_n = e_n \text{ in } t)$$

we have

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x], a_1 :_\pi B_1 = e_1, \ldots, a_n :_\pi B_n = e_n; t) \Downarrow (\Delta_1; v_1)}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x]; \text{let}_\pi\ a_1 : B_1 = e_1, \ldots, a_n : B_n = e_n \text{ in } t) \Downarrow (\Delta_1; v_1)}\ \textit{s-let}_{\mathcal{L}}$$

and

$$\frac{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2], a_1 :_\pi B_1 = e_1, \ldots, a_n :_\pi B_n = e_n; t) \Downarrow (\Delta_2; v_2)}{(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2]; \text{let}_\pi\ a_1 : B_1 = e_1, \ldots, a_n : B_n = e_n \text{ in } t) \Downarrow (\Delta_2; v_2)}\ \textit{s-let}_{\mathcal{L}}$$

Since we know that

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto x], a_1 :_\pi B_1 = e_1, \ldots, a_n :_\pi B_n = e_n; t) \Downarrow (\Delta_1; v_1)$$

then, by induction hypothesis, there is

$$(\Gamma, x :_{\pi_1} A_1 = t_2, y :_{\pi_2} A_2 = e[z \mapsto t_2], a_1 :_\pi B_1 = e_1, \ldots, a_n :_\pi B_n = e_n; t) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$.

Lemma 6 (Γ Substitution Lemma).
If $(\Gamma, a :_\pi A = t_2; t_1[y \mapsto a]) \Downarrow (\Gamma_1; \sigma_1)$ *and* $a \notin fv(t_1)$, *then*

$$(\Gamma; t_1[y \mapsto t_2]) \Downarrow (\Gamma_2; \sigma_2) \textit{ and } deref_{\Gamma_1}(\sigma_1) = deref_{\Gamma_2}(\sigma_2)$$

<u>Proof</u> By induction on $(\Gamma; t) \Downarrow (\Gamma'; t')$:

- Given $(\Gamma, a :_\omega A = t_2; x[y \mapsto a])$, there are two cases:
  - $x = y$
    Since $x[y \mapsto a] = a$, we have
    $$\frac{(\Gamma; t_2) \Downarrow (\Delta; v_1)}{(\Gamma, a :_\omega A = t_2; a) \Downarrow (\Delta, a :_\omega A = v_1; v_1)}\ \textit{s-unvar}_{\mathcal{L}}$$
    and, since $x[y \mapsto t_2] = t_2$ we already know that $(\Gamma; t_2) \Downarrow (\Delta; v_1)$ and we can conclude that $a \notin fv(v_1)$ so, by proposition ,
    $$deref_{\Delta, a:_\omega A = v_1}(v_1) = deref_\Delta(v_1)$$
  - $x \neq y$
    Since $x[y \mapsto a] = x$, we have $(\Gamma, a :_\omega A = t_2; x) \Downarrow (\Delta; v)$ and $a \notin fv(x)$ so, by proposition 2, we can conclude that, since $x[y \mapsto t_2] = x$,
    $$(\Gamma; x) \Downarrow (\Delta; v)$$
    Therefore, the property holds.
- Given $(\Gamma, a :_1 A = t_2; x[y \mapsto a])$, there are two cases:
  - $x = y$
    Since $x[y \mapsto a] = a$, we have
    $$\frac{(\Gamma; t_2) \Downarrow (\Delta; v_2)}{(\Gamma, a :_1 A = t_2; a) \Downarrow (\Delta; v_2)}\ \textit{s-linvar}_{\mathcal{L}}$$
    and, since $x[y \mapsto t_2] = t_2$ we already know that $(\Gamma; t_2) \Downarrow (\Delta; v_2)$. So, the property holds.
  - $x \neq y$
    Since $x[y \mapsto a] = x$, we have $(\Gamma, a :_1 A = t_2; x) \Downarrow (\Delta; v)$ and $a \notin fv(x)$ so, by proposition 2, we can conclude that, since $x[y \mapsto t_2] = x$,
    $$(\Gamma; x) \Downarrow (\Delta; v)$$
- Given $(\Gamma, a :_{\pi_2} A = t_2; \lambda_{\pi_1} x : A.\ t)$, there are two cases
  - $x = y$
    Since $(\lambda_{\pi_1} x : A.\ t)[y \mapsto a] = \lambda_{\pi_1} x : A.$ t, we have
    $$\frac{}{(\Gamma, a :_{\pi_2} B = t_2; \lambda_{\pi_1} x : A.\ t) \Downarrow (\Gamma, a :_{\pi_2} B = t_2; \lambda_{\pi_1} x : A.\ t)}\ \textit{s-abs}_{\mathcal{L}}$$
    and, since $(\lambda_{\pi_1} x : A.\ t)[y \mapsto t_2] = \lambda_{\pi_1} x : A.$ t, we have
    $$\frac{}{(\Gamma; \lambda_{\pi_1} x : A.\ t) \Downarrow (\Gamma; \lambda_{\pi_1} x : A.\ t)}\ \textit{s-abs}_{\mathcal{L}}$$
    Since we know that $a \notin fv(\lambda_{\pi_1} x : A.\ t)$, then
    $$deref_{\Gamma, a:_{\pi_2} B = t_2}(\lambda_{\pi_1} x : A.\ t) = deref_\Gamma(\lambda_{\pi_1} x : A.\ t)$$
    So, the property holds.

- $x \neq y$
  Since $(\lambda_{\pi_1} x : A.\ t)[y \mapsto a] = \lambda_{\pi_1} x : A.\ t[y \mapsto a]$, we have, by rule *s-abs*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma, a :_{\pi_2} B = t_2; \lambda_{\pi_1} x : A.\ t[y \mapsto a])\\ \Downarrow\ &(\Gamma, a :_{\pi_2} B = t_2; \lambda_{\pi_1} x : A.\ t[y \mapsto a])\end{aligned}$$

  and, since $(\lambda_{\pi_1} x : A.\ t)[y \mapsto t_2] = \lambda_{\pi_1} x : A.\ t[y \mapsto t_2]$, we have

$$\frac{}{(\Gamma; \lambda_{\pi_1} x : A.\ t[y \mapsto t_2]) \Downarrow (\Gamma; \lambda_{\pi_1} x : A.\ t[y \mapsto t_2])}\ \textit{s-abs}_{\mathcal{L}}$$

  Now, we have

$$\begin{aligned}&\mathit{deref}_{\Gamma, a:_{\pi_2} B = t_2}(\lambda_{\pi_1} x : A.\ t[y \mapsto a])\\ =\ &\text{let } (v, \Gamma') = \mathit{deref}_{\Gamma, a:_{\pi_2} B = t_2}(t[y \mapsto a])\\ &\qquad \text{by proposition 3}\\ &\qquad = \mathit{deref}_{\Gamma}(t[y \mapsto t_2])\\ &\text{in } (\lambda_{\pi_1} x : A.\ v, \Gamma')\end{aligned}$$

  Since we already know that $(v, \Gamma') = \mathit{deref}_{\Gamma}(t[y \mapsto t_2])$ then, we can conclude that

$$\mathit{deref}_{\Gamma}(\lambda_{\pi_1} x : A.\ t[y \mapsto t_2]) = (\lambda_{\pi_1} x : A.\ v, \Gamma')$$

  and the property holds.

– Given $(\Gamma, a :_{\pi_1} A = t_2; < y_1, y_2 >_{\pi})$,
  Since $(< y_1, y_2 >_{\pi})[y \mapsto a] =< y_1[y \mapsto a], y_2[y \mapsto a] >_{\pi}$, we have, by rule *s-pair*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma, a :_{\pi_1} B = t_2; < y_1[y \mapsto a], y_2[y \mapsto a] >_{\pi})\\ \Downarrow\ &(\Gamma, a :_{\pi_1} B = t_2; < y_1[y \mapsto a], y_2[y \mapsto a] >_{\pi})\end{aligned}$$

  and, since $(< y_1, y_2 >_{\pi})[y \mapsto t_2] =< y_1[y \mapsto t_2], y_2[y \mapsto t_2] >_{\pi}$, we have, by rule *s-pair*$_{\mathcal{L}}$

$$\begin{aligned}&(\Gamma; < y_1[y \mapsto t_2], y_2[y \mapsto t_2] >_{\pi})\\ \Downarrow\ &(\Gamma; < y_1[y \mapsto t_2], y_2[y \mapsto t_2] >_{\pi})\end{aligned}$$

  Now, we have

$$\begin{aligned}&\mathit{deref}_{\Gamma, a:_{\pi_1} B = t_2}(< y_1[y \mapsto a], y_2[y \mapsto a] >_{\pi})\\ =\ &\text{let } (v_1, \Gamma_1) = \mathit{deref}_{\Gamma, a:_{\pi_1} B = t_2}(y_1[y \mapsto a])\\ &\qquad \text{by proposition 3,}\\ &\qquad = \mathit{deref}_{\Gamma}(y_1[y \mapsto t_2])\\ &\quad (v_2, \Gamma_2) = \mathit{deref}_{\Gamma_1, a:_{\pi_1} B = t_2}(y_2[y \mapsto a])\\ &\qquad \text{by proposition 3,}\\ &\qquad = \mathit{deref}_{\Gamma_1}(y_2[y \mapsto t_2])\\ &\text{in } (< v_1, v_2 >_{\pi}, \Gamma_2)\end{aligned}$$

and

$$deref_{\Gamma}(< y_1[y \mapsto t_2], y_2[y \mapsto t_2] >_{\pi})$$

Since we know that

$$(v_1, \Gamma_1) = deref_{\Gamma}(y_1[y \mapsto t_2]) \text{ and } (v_2, \Gamma_2) = deref_{\Gamma_1}(y_2[y \mapsto t_2])$$

Then the property holds.

– Given $(\Gamma, a :_{\pi} A = t_2; e\ x)$, there are two cases
  - $x = y$
    Since $(e\ x)[y \mapsto a] = e[y \mapsto a]\ x[y \mapsto a] = e[y \mapsto a]\ a$, we have

$$\frac{(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]) \Downarrow (\Gamma'; \lambda_{\pi_2} z_1 : B.\ t) \quad (\Gamma'; t[z_1 \mapsto a]) \Downarrow (\Delta_1; v_1)}{(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]\ a) \Downarrow (\Delta_1; v_1)}\ s\text{-}app_{\mathcal{L}}$$

    We know that $(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]) \Downarrow (\Gamma'; \lambda_{\pi_2} z_1 : B.\ t)$ then, by induction hypothesis, $(\Gamma; e[y \mapsto t_2]) \Downarrow (\Gamma''; \lambda_{\pi_2} z_2 : B.\ t')$ and

$$deref_{\Gamma'}(\lambda_{\pi_2} z_1 : B.\ t) = deref_{\Gamma''}(\lambda_{\pi_2} z_2 : B.\ t')$$

    and we can conclude that $deref_{\Gamma'}(t) = deref_{\Gamma''}(t')$.
    We also know that

$$\begin{aligned}(e\ x)[y \mapsto t_2] &= e[y \mapsto t_2]\ x[y \mapsto t_2] \\ &= e[y \mapsto t_2]\ t_2\end{aligned}$$

    so, by rule $app_{\mathcal{L}}$, we have

$$\frac{(\Gamma; e[y \mapsto t_2]) \Downarrow (\Gamma''; \lambda_{\pi_2} z_2 : B.\ t') \quad (\Gamma''; t'[z_2 \mapsto t_2]) \Downarrow (\Delta_2; v_2)}{(\Gamma; e[y \mapsto t_2]\ t_2) \Downarrow (\Delta_2; v_2)}$$

    Since we know that $deref_{\Gamma'}(t) = deref_{\Gamma''}(t')$ then, by induction hypothesis, if $(\Gamma'; t[z_1 \mapsto a]) \Downarrow (\Delta_1; v_1)$ then, by α-conversion,

$$(\Gamma''; t'[z_2 \mapsto t_2]) \Downarrow (\Delta_2; v_2)$$

    and $deref_{\Delta_1}(v_1) = deref_{\Delta_2}(v_2)$. So the property holds.
  - $x \neq y$
    Since $(e\ x)[y \mapsto a] = e[y \mapsto a]\ x$, we have

$$\frac{(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]) \Downarrow (\Gamma'; \lambda_{\pi_2} z_1 : B.\ t) \quad (\Gamma'; t[z_1 \mapsto x]) \Downarrow (\Delta_1; v_1)}{(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]\ x) \Downarrow (\Delta_1; v_1)}\ s\text{-}app_{\mathcal{L}}$$

    We know that $(\Gamma, a :_{\pi_1} A = t_2; e[y \mapsto a]) \Downarrow (\Gamma'; \lambda_{\pi_2} z_1 : B.\ t)$ then, by induction hypothesis, $(\Gamma; e[y \mapsto t_2]) \Downarrow (\Gamma''; \lambda_{\pi_2} z_2 : B.\ t')$ and

$$deref_{\Gamma'}(\lambda_{\pi_2} z_1 : B.\ t) = deref_{\Gamma''}(\lambda_{\pi_2} z_2 : B.\ t')$$

    and we can conclude that $deref_{\Gamma'}(t) = deref_{\Gamma''}(t')$.

We also know that

$$\begin{aligned}(e\ x)[y \mapsto t_2] &= e[y \mapsto t_2]\ x[y \mapsto t_2] \\ &= e[y \mapsto t_2]\ x\end{aligned}$$

so, by rule *s-app*$_{\mathcal{L}}$,

$$\frac{\begin{array}{c}(\Gamma; e[y \mapsto t_2]) \Downarrow (\Gamma''; \lambda_{\pi_2} z_2 : B.\ t') \\ (\Gamma''; t'[z_2 \mapsto x]) \Downarrow (\Delta_2; v_2)\end{array}}{(\Gamma; e[y \mapsto t_2]\ x) \Downarrow (\Delta_2; v_2)}$$

Since we know that $deref_{\Gamma'}(t) = deref_{\Gamma''}(t')$ then, by induction hypothesis, if $(\Gamma'; t[z_1 \mapsto x]) \Downarrow (\Delta_1; v_1)$ then, by α-conversion,

$$(\Gamma''; t'[z_2 \mapsto x]) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$, so the property holds.

– Given $(\Gamma, a :_{\pi_1} A = t_2; \text{split } e_1 \text{ as } y_1, z_1 \text{ in } e_2)$, since

$$(\text{split } e_1 \text{ as } y_1, z_1 \text{ in } e_2)[y \mapsto a] = \text{split } e_1[y \mapsto a] \text{ as } y_1, z_1 \text{ in } e_2[y \mapsto a]$$

we have

$$\frac{\begin{array}{c}(\Gamma, a :_{\pi_1} A = t_2; e_1[y \mapsto a]) \Downarrow (\Gamma'; < y_2, z_2 >_{\pi_2}) \\ (\Gamma'; e_2[y \mapsto a][y_1 \mapsto y_2][z_1 \mapsto z_2]) \Downarrow (\Delta_1; v_1)\end{array}}{(\Gamma, a :_{\pi_1} A = t_2; \text{split } e_1[y \mapsto a] \text{ as } y_1, z_1 \text{ in } e_2[y \mapsto a]) \Downarrow (\Delta_1; v_1)}\ \textit{s-split}_{\mathcal{L}}$$

We know that $(\Gamma, a :_{\pi_1} A = t_2; e_1[y \mapsto a]) \Downarrow (\Gamma'; < y_2, z_2 >_{\pi_2})$ then, by induction hypothesis, $(\Gamma; e_1[y \mapsto t_2]) \Downarrow (\Gamma''; < y_3, z_3 >_{\pi_2})$ and

$$deref_{\Gamma'}(< y_2, z_2 >_{\pi_2}) = deref_{\Gamma''}(< y_3, z_3 >_{\pi_2})$$

We also know that

$$(\text{split } e_1 \text{ as } y_1, z_1 \text{ in } e_2)[y \mapsto t_2] = \text{split } e_1[y \mapsto t_2] \text{ as } y_1, z_1 \text{ in } e_2[y \mapsto t_2]$$

so, by rule *s-split*$_{\mathcal{L}}$,

$$\frac{\begin{array}{c}(\Gamma; e_1[y \mapsto t_2]) \Downarrow (\Gamma''; < y_3, z_3 >_{\pi_2}) \\ (\Gamma''; e_2[y \mapsto t_2][y_1 \mapsto y_3][z_1 \mapsto z_3]) \Downarrow (\Delta_2; v_2)\end{array}}{(\Gamma; \text{split } e_1[y \mapsto t_2] \text{ as } y_1, z_1 \text{ in } e_2[y \mapsto t_2]) \Downarrow (\Delta_2; v_2)}$$

Since we know that $deref_{\Gamma'}(< y_2, z_2 >_{\pi_2}) = deref_{\Gamma''}(< y_3, z_3 >_{\pi_2})$, if $(\Gamma'; e_2[y \mapsto a][y_1 \mapsto y_2][z_1 \mapsto z_2]) \Downarrow (\Delta_1; v_1)$ then, by induction hypothesis,

$$(\Gamma''; e_2[y \mapsto t_2][y_1 \mapsto y_3][z_1 \mapsto z_3]) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$. So the property holds.

– Given $(\Gamma, a :_{\pi_2} A = t_2; \mathrm{let}_{\pi_1}\ a_1 : B_1, \ldots,\ a_n : B_n \text{ in } e$,
since

$$\begin{aligned}&(\mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1, \ldots,\ a_n : B_n = e_n \text{ in } t)[y \mapsto a]\\ =\ &\mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1[y \mapsto a], \ldots,\ a_n : B_n = e_n[y \mapsto a] \text{ in } t[y \mapsto a]\end{aligned}$$

we have

$$\frac{(\Gamma, a :_{\pi_2} A = t_2, a_1 :_{\pi_1} B_1 = e_1[y \mapsto a], \ldots,\ a_n :_{\pi_1} B_n = e_n[y \mapsto a]; t[y \mapsto a]) \Downarrow (\Delta_1; v_1)}{(\Gamma, a :_{\pi_2} A = t_2; \mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1[y \mapsto a], \ldots,\ a_n : B_n = e_n[y \mapsto a] \text{ in } t[y \mapsto a]) \Downarrow (\Delta_1; v_1)}\ \textit{s-let}_{\mathcal{L}}$$

by lemma 5, if

$$\begin{aligned}&(\Gamma, a :_{\pi_2} A = t_2, a_1 : B_1 = e_1[y \mapsto a], \ldots,\ a_n : B_n = e_n[y \mapsto a]; t[y \mapsto a])\\ &\Downarrow (\Delta_1; v_1)\end{aligned}$$

then

$$\begin{aligned}&(\Gamma, a :_{\pi_2} A = t_2, a_1 : B_1 = e_1[y \mapsto t_2], \ldots,\ a_n : B_n = e_n[y \mapsto t_2]; t[y \mapsto a])\\ &\Downarrow (\Delta_1'; v_1')\end{aligned}$$

and $deref_{\Delta_1}(v_1) = deref_{\Delta_1{}'}(v_1{}')$. So, by induction hypothesis,

$$(\Gamma, a_1 : B_1 = e_1[y \mapsto t_2], \ldots,\ a_n : B_n = e_n[y \mapsto t_2]; t[y \mapsto t_2]) \Downarrow (\Delta_2; v_2)$$

and $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$. We also know that

$$\begin{aligned}&(\mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1, \ldots,\ a_n : B_n = e_n \text{ in } t)[y \mapsto t_2]\\ =\ &\mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1[y \mapsto t_2], \ldots,\ a_n : B_n = e_n[y \mapsto t_2] \text{ in } t[y \mapsto t_2]\end{aligned}$$

so, by rule *s-let*$_{\mathcal{L}}$,

$$\frac{(\Gamma, a_1 :_{\pi_1} B_1 = e_1[y \mapsto t_2], \ldots,\ a_n :_{\pi_1} B_n = e_n[y \mapsto t_2]; t[y \mapsto t_2]) \Downarrow (\Delta_2; v_2)}{(\Gamma; \mathrm{let}_{\pi_1}\ a_1 : B_1 = e_1[y \mapsto t_2], \ldots,\ a_n : B_n = e_n[y \mapsto t_2] \text{ in } t[y \mapsto t_2]) \Downarrow (\Delta_2; v_2)}$$

Since we already know that $deref_{\Delta_2}(v_2) = deref_{\Delta_1}(v_1)$, the property holds.

Lemma 7 (Substitution Lemma).
Given $M_{\mathcal{W}}[y \mapsto t]$, then $(M_{\mathcal{W}}[y \mapsto t])^*_{\mathcal{L}} = (M_{\mathcal{W}})^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$.

<u>Proof</u> By induction on $M_{\mathcal{W}}$:

– $M_{\mathcal{W}} \equiv x$
There are two cases:
- $x = y$
  We have $(x[y \mapsto t])^*_{\mathcal{L}} = t^*_{\mathcal{L}}$ and $x^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] = x[y \mapsto t^*_{\mathcal{L}}] = t^*_{\mathcal{L}}$. So the property holds.

- $x \neq y$
  We have $(x[y \mapsto t])^*_{\mathcal{L}} = x^*_{\mathcal{L}} = x$ and $x^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] = x[y \mapsto t^*_{\mathcal{L}}] = x$. So the property holds.

– $M_{\mathcal{W}} = q\ \lambda x : q'\ P.\ t_1$
  There are two cases:
  - $x = y$
    We have $((q\ \lambda x : q'\ P.\ t_1)[y \mapsto t])^*_{\mathcal{L}} = (q\ \lambda x : q'\ P.\ t_1)^*_{\mathcal{L}} = \lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}}$, where $\pi = q'_{\mathcal{L}}$, and

    $$(q\ \lambda x : q'\ P.\ t_1)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] = (\lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}})[y \mapsto t^*_{\mathcal{L}}] = \lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}}$$

    So the property holds.
  - $x \neq y$
    We have

    $$((q\ \lambda x : q'\ P.\ t_1)[y \mapsto t])^*_{\mathcal{L}} = (q\ \lambda x : q'\ P.\ t_1[y \mapsto t])^*_{\mathcal{L}} = \lambda_\pi x : P_{\mathcal{L}}.(t_1[y \mapsto t])^*_{\mathcal{L}}$$

    where $\pi = \mathrm{q}'_{\mathcal{L}}$, and

    $$(q\ \lambda x : q'\ P.\ t_1)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] = (\lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}})[y \mapsto t^*_{\mathcal{L}}] = \lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$$

    By induction hypothesis, we know that $(t_1[y \mapsto t])^*_{\mathcal{L}} = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

    $$\lambda_\pi x : P_{\mathcal{L}}.(t_1[y \mapsto t])^*_{\mathcal{L}} = \lambda_\pi x : P_{\mathcal{L}}.{t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$$

– $M_{\mathcal{W}} = q < t_1, t_2 >$
  We have

  $$\begin{aligned} & ((q\ \ < t_1, t_2 >)[y \mapsto t])^*_{\mathcal{L}} \\ = & (q\ \ < t_1[y \mapsto t], t_2[y \mapsto t] >)^*_{\mathcal{L}} \\ = & \ \mathrm{let}_\pi\ a_1 : A_1 = (t_1[y \mapsto t])^*_{\mathcal{L}}, a_2 : A_2 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ \ < a_1, a_2 >_\pi \\ & \text{where } \pi = q_{\mathcal{L}} \end{aligned}$$

  and

  $$\begin{aligned} & (q\ \ < t_1, t_2 >)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\ = & (\mathrm{let}_\pi\ b_1 : B_1 = {t_1}^*_{\mathcal{L}}, b_2 : B_2 = {t_2}^*_{\mathcal{L}}\ \mathrm{in}\ \ < b_1, b_2 >_\pi)[y \mapsto t^*_{\mathcal{L}}] \\ = & \ \mathrm{let}_\pi\ b_1 : B_1 = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}], b_2 : B_2 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ (< b_1, b_2 >_\pi)[y \mapsto t^*_{\mathcal{L}}] \\ = & \ \mathrm{let}_\pi\ b_1 : B_1 = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}], b_2 : B_2 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ \ < b_1, b_2 >_\pi \end{aligned}$$

  By induction hypothesis, we know that $(t_1[y \mapsto t])^*_{\mathcal{L}} = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$ and $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

  $$\begin{aligned} & \mathrm{let}_\pi\ a_1 : A_1 = (t_1[y \mapsto t])^*_{\mathcal{L}}, a_2 : A_2 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ \ < a_1, a_2 >_\pi \\ = & \ \mathrm{let}_\pi\ b_1 : B_1 = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}], b_2 : B_2 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ \ < b_1, b_2 >_\pi \end{aligned}$$

– $M_{\mathcal{W}} \equiv x\ t_1$
  There are two cases:
  - $x = y$
    We have

$$\begin{aligned}((x\ t_2)[y \mapsto t])^*_{\mathcal{L}} &= (x[y \mapsto t]\ t_2[y \mapsto t])^*_{\mathcal{L}} \\ &= (t\ t_2[y \mapsto t])^*_{\mathcal{L}} \\ &= \mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ t^*_{\mathcal{L}}\ a_1\end{aligned}$$

    and

$$\begin{aligned}&(x\ t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\ &= (\mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}\ \mathrm{in}\ x^*_{\mathcal{L}}\ b_1)[y \mapsto t^*_{\mathcal{L}}] \\ &= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ x[y \mapsto t^*_{\mathcal{L}}]\ b_1[y \mapsto t^*_{\mathcal{L}}] \\ &= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ t^*_{\mathcal{L}}\ b_1\end{aligned}$$

    By induction hypothesis, we know that $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ t^*_{\mathcal{L}}\ a_1 = \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ t^*_{\mathcal{L}}\ b_1$$

  - $x \neq y$
    We have

$$\begin{aligned}((x\ t_2)[y \mapsto t])^*_{\mathcal{L}} &= (x[y \mapsto t]\ t_2[y \mapsto t])^*_{\mathcal{L}} \\ &= (x\ t_2[y \mapsto t])^*_{\mathcal{L}} \\ &= \mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ x\ a_1\end{aligned}$$

    and

$$\begin{aligned}&(x\ t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\ &= (\mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}\ \mathrm{in}\ x^*_{\mathcal{L}}\ b_1)[y \mapsto t^*_{\mathcal{L}}] \\ &= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ x[y \mapsto t^*_{\mathcal{L}}]\ b_1[y \mapsto t^*_{\mathcal{L}}] \\ &= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ x\ b_1\end{aligned}$$

    By induction hypothesis, we know that $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ x\ a_1 = \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ x\ b_1$$

– $M_{\mathcal{W}} = t_1\ t_2$
  We have

$$\begin{aligned}((t_1\ t_2)[y \mapsto t])^*_{\mathcal{L}} &= (t_1[y \mapsto t] t_2[y \mapsto t])^*_{\mathcal{L}} \\ &= \mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ (t_1[y \mapsto t])^*_{\mathcal{L}}\ a_1\end{aligned}$$

and

$$\begin{aligned}
&(t_1\ t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\
&= (\mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}\ \mathrm{in}\ {t_1}^*_{\mathcal{L}}\ b_1)[y \mapsto t^*_{\mathcal{L}}] \\
&= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ b_1[y \mapsto t^*_{\mathcal{L}}] \\
&= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ b_1
\end{aligned}$$

By induction hypothesis, we know that $(t_1[y \mapsto t])^*_{\mathcal{L}} = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$ and $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\begin{aligned}
&\mathrm{let}_\pi\ a_1 : A_1 = (t_2[y \mapsto t])^*_{\mathcal{L}}\ \mathrm{in}\ (t_1[y \mapsto t])^*_{\mathcal{L}}\ a_1 \\
&= \mathrm{let}_\pi\ b_1 : B_1 = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{in}\ {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]\ b_1
\end{aligned}$$

– $M_{\mathcal{W}} = \mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2$
There are two cases:

- $x = y$
  We have

$$\begin{aligned}
&((\mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2)[y \mapsto t])^*_{\mathcal{L}} \\
&= (\mathrm{split}\ x_1[y \mapsto t]\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= (\mathrm{split}\ t\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= \mathrm{split}\ t^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ (t_2[y \mapsto t])^*_{\mathcal{L}}
\end{aligned}$$

  and

$$\begin{aligned}
&(\mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\
&= (\mathrm{split}\ {x_1}^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ {t_2}^*_{\mathcal{L}})[y \mapsto t^*_{\mathcal{L}}] \\
&= \mathrm{split}\ x_1[y \mapsto t^*_{\mathcal{L}}]\ \mathrm{as}\ y, z\ \mathrm{in}\ {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\
&= \mathrm{split}\ t^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]
\end{aligned}$$

  By induction hypothesis, we know that $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\mathrm{split}\ t^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ (t_2[y \mapsto t])^*_{\mathcal{L}} = \mathrm{split}\ t^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$$

- $x \neq y$
  We have

$$\begin{aligned}
&((\mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2)[y \mapsto t])^*_{\mathcal{L}} \\
&= (\mathrm{split}\ x_1[y \mapsto t]\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= (\mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= \mathrm{split}\ {x_1}^*_{\mathcal{L}}\ \mathrm{as}\ y, z\ \mathrm{in}\ (t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= \mathrm{split}\ x_1\ \mathrm{as}\ y, z\ \mathrm{in}\ (t_2[y \mapsto t])^*_{\mathcal{L}}
\end{aligned}$$

and

$$\begin{aligned}
&(\text{split } x_1 \text{ as } y, z \text{ in } t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\
&= (\text{split } {x_1}^*_{\mathcal{L}} \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}})[y \mapsto t^*_{\mathcal{L}}] \\
&= \text{split } x_1[y \mapsto t^*_{\mathcal{L}}] \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \\
&= \text{split } x_1 \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]
\end{aligned}$$

By induction hypothesis, we know that $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\text{split } x_1 \text{ as } y, z \text{ in } (t_2[y \mapsto t])^*_{\mathcal{L}} = \text{split } x_1 \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$$

– $M_{\mathcal{L}} = \text{split } t_1 \text{ as } y, z \text{ in } t_2$
We have

$$\begin{aligned}
((\text{split } t_1 \text{ as } y, z \text{ in } t_2)[y \mapsto t])^*_{\mathcal{L}} &= (\text{split } t_1[y \mapsto t] \text{ as } y, z \text{ in } t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= \text{split } (t_1[y \mapsto t])^*_{\mathcal{L}} \text{ as } y, z \text{ in } (t_2[y \mapsto t])^*_{\mathcal{L}}
\end{aligned}$$

and

$$\begin{aligned}
(\text{split } t_1 \text{ as } y, z \text{ in } t_2)^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] &= (\text{split } {t_1}^*_{\mathcal{L}} \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}})[y \mapsto t^*_{\mathcal{L}}] \\
&= \text{split } {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]
\end{aligned}$$

By induction hypothesis, we know that $(t_1[y \mapsto t])^*_{\mathcal{L}} = {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$ and $(t_2[y \mapsto t])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]$, so we can conclude that

$$\begin{aligned}
&\text{split } (t_1[y \mapsto t])^*_{\mathcal{L}} \text{ as } y, z \text{ in } (t_2[y \mapsto t])^*_{\mathcal{L}} \\
&= \text{split } {t_1}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}] \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}[y \mapsto t^*_{\mathcal{L}}]
\end{aligned}$$

Lemma 8. *Given $(\Gamma; M_{\mathcal{L}})$, then $deref_{\Gamma_{\mathcal{W}}}((M_{\mathcal{L}}{}^*)_{\mathcal{W}}) = deref_{\Gamma_{\mathcal{W}}}((M_{\mathcal{L}})_{\mathcal{W}})$.*

<u>Proof</u> By induction on $M_{\mathcal{L}}$:

– Given $(\Gamma, x :_1 A = t; x)$, we have

$$(\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}; x_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}; x)$$

and

$$\begin{aligned}
(\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}; (x^*)_{\mathcal{W}}) &= (\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}; x_{\mathcal{W}}) \\
&= (\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}; x)
\end{aligned}$$

Then we have $deref_{\Gamma_{\mathcal{W}}, x \mapsto \text{lin } t_{\mathcal{W}}}(x) = deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}})$. So the property holds.

– Given $(\Gamma, x :_\omega A = t; x)$, we have

$$(\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}; x_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}; x)$$

and

$$\begin{aligned}
(\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}; (x^*)_{\mathcal{W}}) &= (\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}; x_{\mathcal{W}}) \\
&= (\Gamma_{\mathcal{W}}, x \mapsto \text{ un } t_{\mathcal{W}}; x)
\end{aligned}$$

Then we have $deref_{\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}}(x) = deref_{\Gamma_{\mathcal{W}}, x \mapsto \text{un } t_{\mathcal{W}}}(t_{\mathcal{W}})$. So the property holds.

– Given $(\Gamma; \lambda_\pi y : P.t)$, we have $(\Gamma_\mathcal{W}; (\lambda_\pi y : P.t)_\mathcal{W}) = (\Gamma_\mathcal{W}; \pi_\mathcal{W} \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.t_\mathcal{W})$ and

$$\begin{aligned}(\Gamma_\mathcal{W}; ((\lambda_\pi y : P.t)^*)_\mathcal{W}) &= (\Gamma_\mathcal{W}; (\lambda_\pi y : P.t^*)_\mathcal{W}) \\ &= (\Gamma_\mathcal{W}; \pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.(t^*)_\mathcal{W})\end{aligned}$$

We have

$$\begin{aligned}& dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.t_\mathcal{W}) \\ =\ & \text{let } (v, S) = dereff_{\Gamma_\mathcal{W}}(t_\mathcal{W}) \\ & \text{in } (\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.\ v, S)\end{aligned}$$

and

$$\begin{aligned}& dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.(t^*)_\mathcal{W}) \\ =\ & \text{let } (v', S') = dereff_{\Gamma_\mathcal{W}}((t^*)_\mathcal{W}) \\ & \text{in } (\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.\ v', S')\end{aligned}$$

By induction hypothesis, $dereff_{\Gamma_\mathcal{W}}(t_\mathcal{W}) = dereff_{\Gamma_\mathcal{W}}((t^*)_\mathcal{W})$. Therefore, we have $dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.t_\mathcal{W}) = dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W}\ \lambda y : \pi_\mathcal{W}\ P_\mathcal{W}.(t^*)_\mathcal{W})$, so the property holds.

– Given $(\Gamma; < t_1, t_2 >_\pi)$, we have $(\Gamma_\mathcal{W}; \pi_\mathcal{W} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >)$ and

$$\begin{aligned}& (\Gamma_\mathcal{W}; ((< t_1, t_2 >_\pi)^*)_\mathcal{W}) \\ =\ & (\Gamma_\mathcal{W}; (\text{let}_\pi\ a_1 : A_1 = {t_1}^*, a_2 : A_2 = {t_2}^* \text{ in } < a_1, a_2 >_\pi)_\mathcal{W}) \\ =\ & (\Gamma_\mathcal{W}; (< a_1, a_2 >_\pi)_\mathcal{W}[a_1 \mapsto ({t_1}^*)_\mathcal{W}][a_2 \mapsto ({t_2}^*)_\mathcal{W}]) \\ =\ & (\Gamma_\mathcal{W}; (\pi_\mathcal{W} < a_1, a_2 >)[a_1 \mapsto ({t_1}^*)_\mathcal{W}][a_2 \mapsto ({t_2}^*)_\mathcal{W}]) \\ & \text{since we know that } a_1 \notin fv(({t_2}^*)_\mathcal{W}) \text{ and } a_2 \notin fv(({t_1}^*)_\mathcal{W}) \\ =\ & (\Gamma_\mathcal{W}; \pi_\mathcal{W} < a_1[a_1 \mapsto ({t_1}^*)_\mathcal{W}], a_2[a_2 \mapsto ({t_2}^*)_\mathcal{W}] >) \\ =\ & (\Gamma_\mathcal{W}; \pi_\mathcal{W} < ({t_1}^*)_\mathcal{W}, ({t_2}^*)_\mathcal{W} >)\end{aligned}$$

We now know that

$$\begin{aligned}& dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >) \\ =\ & \text{let } (v_1, S_1) = dereff_{\Gamma_\mathcal{W}}(t_{1\mathcal{W}}) \\ & \quad\ (v_2, S_2) = dereff_{S_1}(t_{2\mathcal{W}}) \\ & \text{in } (\pi_\mathcal{W} < v_1, v_2 >, S_2)\end{aligned}$$

and

$$\begin{aligned}& dereff_{\Gamma_\mathcal{W}}(\pi_\mathcal{W} < ({t_1}^*)_\mathcal{W}, ({t_2}^*)_\mathcal{W} >) \\ =\ & \text{let } ({v_1}', {S_1}') = dereff_{\Gamma_\mathcal{W}}(({t_1}^*)_\mathcal{W}) \\ & \quad\ ({v_2}', {S_2}') = dereff_{{S_1}'}(({t_2}^*)_\mathcal{W}) \\ & \text{in } (\pi_\mathcal{W} < {v_1}', {v_2}' >, {S_2}')\end{aligned}$$

By induction hypothesis,

$$deref_{\Gamma_{\mathcal{W}}}(t_1)_{\mathcal{W}} = deref_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}})$$

Then we know that $S_1 = S_1{}'$. So, similarly, by induction hypothesis

$$deref_{S_1}(t_{2\mathcal{W}}) = deref_{S_1{}'}((t_2{}^*)_{\mathcal{W}})$$

Therefore, we can conclude that

$$deref_{\Gamma_{\mathcal{W}}}(\pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >) = deref_{\Gamma_{\mathcal{W}}}(\pi_{\mathcal{W}} < (t_1{}^*)_{\mathcal{W}}, (t_2{}^*)_{\mathcal{W}} >)$$

The property holds.

- Given $(\Gamma; t\ x)$, we have $(\Gamma_{\mathcal{W}}; (t\ x)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; t_{\mathcal{W}}\ x_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; t_{\mathcal{W}}\ x)$ and $(\Gamma_{\mathcal{W}}; ((t\ x)^*)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; (t^*\ x^*)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; (t^*)_{\mathcal{W}}\ x_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; (t^*)_{\mathcal{W}}\ x)$. We know that

$$\begin{aligned} deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}\ x) = \text{let } & (v_1, S_1) = deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}) \\ & (v_2, S_2) = deref_{S_1}(x) \\ \text{in } & (v_1\ v_2, S_2) \end{aligned}$$

and

$$\begin{aligned} deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}}\ x) = \text{let } & (v_1{}', S_1{}') = deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}}) \\ & (v_2{}', S_2{}') = deref_{S_1{}'}(x) \\ \text{in } & (v_1{}'\ v_2{}', S_2{}') \end{aligned}$$

By induction hypothesis, $deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}) = deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}})$ and $S_1 = S_1{}'$, therefore, we can conclude that $deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}\ x) = deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}}\ x)$, so the property holds.

- Given $(\Gamma; t_1\ t_2)$, we have $(\Gamma_{\mathcal{W}}; (t_1\ t_2)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; t_{1\mathcal{W}}\ t_{2\mathcal{W}})$ and

$$\begin{aligned} (\Gamma_{\mathcal{W}}; ((t_1\ t_2)^*)_{\mathcal{W}}) &= (\Gamma_{\mathcal{W}}; (\text{let}_\pi\ a : A = t_2{}^*\ \text{in}\ t_1{}^*\ a)_{\mathcal{W}}) \\ &= (\Gamma_{\mathcal{W}}; (t_1{}^*\ a)_{\mathcal{W}}[a \mapsto (t_2{}^*)_{\mathcal{W}}]) \\ &= (\Gamma_{\mathcal{W}}; ((t_1{}^*)_{\mathcal{W}}\ a_{\mathcal{W}})[a \mapsto (t_2{}^*)_{\mathcal{W}}] \\ &\text{we know that } a \notin fv((t_1{}^*)_{\mathcal{W}}) \\ &= (\Gamma_{\mathcal{W}}; (t_1{}^*)_{\mathcal{W}}\ a[a \mapsto (t_2{}^*)_{\mathcal{W}}]) \\ &= (\Gamma_{\mathcal{W}}; (t_1{}^*)_{\mathcal{W}}\ (t_2{}^*)_{\mathcal{W}}) \end{aligned}$$

We know that

$$\begin{aligned} deref_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}\ t_{2\mathcal{W}}) = \text{let } & (v_1, S_1) = deref_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}) \\ & (v_2, S_2) = deref_{S_1}(t_{2\mathcal{W}}) \\ \text{in } & (v_1\ v_2, S_2) \end{aligned}$$

and

$$\begin{aligned} dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}}\ (t_2{}^*)_{\mathcal{W}}) = \text{let } & (v_1{}', S_1{}') = dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}}) \\ & (v_2{}', S_2{}') = dereff_{S_1{}'}((t_2{}^*)_{\mathcal{W}}) \\ \text{in } & (v_1{}'\ v_2{}', S_2{}') \end{aligned}$$

By induction hypothesis, $dereff_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}) = dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}})$ and $S_1 = S_1{}'$, then we also know that $dereff_{S_1}(t_{2\mathcal{W}}) = dereff_{S_1{}'}((t_2{}^*)_{\mathcal{W}})$.
We can then conclude $dereff_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}\ t_{2\mathcal{W}}) = dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}}\ (t_2{}^*)_{\mathcal{W}})$.

– Given $(\Gamma; \text{split } t_1 \text{ as } y, z \text{ in } t_2)$, we have

$$(\Gamma_{\mathcal{W}}; (\text{split } t_1 \text{ as } y, z \text{ in } t_2)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; \text{split } t_{1\mathcal{W}} \text{ as } y, z \text{ in } t_{2\mathcal{W}})$$

and

$$\begin{aligned} (\Gamma_{\mathcal{W}}; ((\text{split } t_1 \text{ as } y, z \text{ in } t_2)^*)_{\mathcal{W}}) &= (\Gamma_{\mathcal{W}}; (\text{split } t_1{}^* \text{ as } y, z \text{ in } t_2{}^*)_{\mathcal{W}}) \\ &= (\Gamma_{\mathcal{W}}; \text{split } (t_1{}^*)_{\mathcal{W}} \text{ as } y, z \text{ in } (t_2{}^*)_{\mathcal{W}}) \end{aligned}$$

We know that

$$\begin{aligned} & dereff_{\Gamma_{\mathcal{W}}}(\text{split } t_{1\mathcal{W}} \text{ as } y, z \text{ in } t_{2\mathcal{W}}) \\ = \text{let } & (v_1, S_1) = dereff_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}) \\ & (v_2, S_2) = dereff_{S_1}(t_{2\mathcal{W}}) \\ \text{in } & (\text{split } v_1 \text{ as } y, z \text{ in } v_2, S_2) \end{aligned}$$

and

$$\begin{aligned} & dereff_{\Gamma_{\mathcal{W}}}(\text{split } (t_1{}^*)_{\mathcal{W}} \text{ as } y, z \text{ in } (t_2{}^*)_{\mathcal{W}}) \\ = \text{let } & (v_1{}', S_1{}') = dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}}) \\ & (v_2{}', S_2{}') = dereff_{S_1{}'}((t_2{}^*)_{\mathcal{W}}) \\ \text{in } & (\text{split } v_1{}' \text{ as } y, z \text{ in } v_2{}', S_2{}') \end{aligned}$$

By induction hypothesis

$$dereff_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}) = dereff_{\Gamma_{\mathcal{W}}}((t_1{}^*)_{\mathcal{W}})$$

and $S_1 = S_1{}'$, then we also know that, by induction hypothesis,

$$dereff_{S_1}(t_{2\mathcal{W}}) = dereff_{S_1{}'}((t_2{}^*)_{\mathcal{W}})$$

We can conclude that

$$dereff_{\Gamma_{\mathcal{W}}}(\text{split } t_{1\mathcal{W}} \text{ as } y, z \text{ in } t_{2\mathcal{W}}) = dereff_{\Gamma_{\mathcal{W}}}(\text{split } (t_1{}^*)_{\mathcal{W}} \text{ as } y, z \text{ in } (t_2{}^*)_{\mathcal{W}})$$

so the property holds.

– Given $(\Gamma; \text{let}_\pi\ a_1 : A_1 = t_1, \ldots, a_n : A_n = t_n \text{ in } t)$, we have

$$\begin{aligned}&(\Gamma_{\mathcal{W}}; (\text{let}_\pi\ a_1 : A_1 = t_1, \ldots, a_n : A_n = t_n \text{ in } t)_{\mathcal{W}})\\ =\ &(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}[a_1 \mapsto t_1{}_{\mathcal{W}}] \ldots [a_n \mapsto t_n{}_{\mathcal{W}}])\end{aligned}$$

and

$$\begin{aligned}&(\Gamma_{\mathcal{W}}; ((\text{let}_\pi\ a_1 : A_1 = t_1, \ldots, a_n : A_n = t_n \text{ in } t)^*)_{\mathcal{W}})\\ =\ &(\Gamma_{\mathcal{W}}; (t^*)_{\mathcal{W}}[a_1 \mapsto ({t_1}^*)_{\mathcal{W}}] \ldots [a_n \mapsto ({t_n}^*)_{\mathcal{W}}])\end{aligned}$$

Now we have

$$deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}[a_1 \mapsto t_1{}_{\mathcal{W}}] \ldots [a_n \mapsto t_n{}_{\mathcal{W}}])$$

and

$$deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}}[a_1 \mapsto ({t_1}^*)_{\mathcal{W}}] \ldots [a_n \mapsto ({t_n}^*)_{\mathcal{W}}])$$

Since we now have $(\Gamma; t[a_1 \mapsto t_1] \ldots [a_n \mapsto t_n])$ then, by induction hypothesis,

$$\begin{aligned}&deref_{\Gamma_{\mathcal{W}}}((t^*)_{\mathcal{W}}[a_1 \mapsto ({t_1}^*)_{\mathcal{W}}] \ldots [a_n \mapsto ({t_n}^*)_{\mathcal{W}}])\\ =\ &deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}[a_1 \mapsto t_1{}_{\mathcal{W}}] \ldots [a_n \mapsto t_n{}_{\mathcal{W}}])\end{aligned}$$

So the property holds.

Lemma 9.
If $(S, a \mapsto q\ (t^*)_{\mathcal{W}}; M) \twoheadrightarrow (S_1; N_1)$ *then*

$$(S, a \mapsto q\ t_{\mathcal{W}}; M) \twoheadrightarrow (S_2; N_2) \text{ and } deref_{S_2}(N_2) = deref_{S_1}(N_1)$$

<u>Proof</u> By induction on $(S; t) \twoheadrightarrow (S'; v')$:

– Given $(S, a \mapsto q\ ({t_1}^*)_{\mathcal{W}}; q_1\ \lambda y : q_2\ P.\ t)$, we have, by rule $s\text{-}abs_{\mathcal{W}}$

$$\begin{aligned}&(S, a \mapsto q\ ({t_1}^*)_{\mathcal{W}}; q_1\ \lambda y : q_2\ P.\ t)\\ \to\ &(S, a \mapsto q\ ({t_1}^*)_{\mathcal{W}}, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t; a_1)\end{aligned}$$

We also know that

$$\frac{}{(S, a \mapsto q\ t_1{}_{\mathcal{W}}; q_1\ \lambda y : q_2\ P.\ t) \to (S, a \mapsto q\ t_1{}_{\mathcal{W}}, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t; a_2)}\ s\text{-}abs_{\mathcal{W}}$$

Then

$$\begin{aligned}&deref_{S, a \mapsto q\ ({t_1}^*)_{\mathcal{W}}, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t}(a_1)\\ =\ &deref_{S, a \mapsto q\ ({t_1}^*)_{\mathcal{W}}}(q_1\ \lambda y : q_2\ P.\ t)\end{aligned}$$

since a_1 is a fresh variable, it can be removed from the heap

We know that $q_1\ \lambda y : q_2\ P.$ t is not evaluated, therefore $a \notin fv(t)$. Then, by the $deref_S$ lemma,

$$deref_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}}}(q_1\ \lambda y : q_2\ P.\ t) = deref_S(q_1\ \lambda y : q_2\ P.\ t)$$

and

$$\begin{aligned}
&deref_{S,a\mapsto q\ t_1{}_{\mathcal{W}},a_2\mapsto q_1\ \lambda y:q_2\ P.\ t}(a_2)\\
&= deref_{S,a\mapsto q\ t_1{}_{\mathcal{W}}}(q_1\ \lambda y : q_2\ P.\ t) \text{ because } a_2 \text{ is a fresh variable}\\
&\quad \text{since } a \notin fv(t), \text{ by the } deref_S \text{ lemma}\\
&= deref_S(q_1\ \lambda y : q_2\ P.\ t)
\end{aligned}$$

The property holds.

– Given $(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; q_1 < e_1, e_2 >)$, we have, by rule *s-pair*$_{\mathcal{W}}$

$$\begin{aligned}
&(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; q_1 < e_1, e_2 >)\\
\rightarrow\ &(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}, a_1 \mapsto q_1 < e_1, e_2 >; a_1)
\end{aligned}$$

We also know that, by rule *s-pair*$_{\mathcal{W}}$

$$\begin{aligned}
&(S, a \mapsto q\ t_1{}_{\mathcal{W}}; q_1 < e_1, e_2 >)\\
\rightarrow\ &(S, a \mapsto q\ t_1{}_{\mathcal{W}}, a_2 \mapsto q_1 < e_1, e_2 >; a_2)
\end{aligned}$$

Then

$$deref_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}},a_1\mapsto q_1<e_1,e_2>}(a_1) = deref_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}}}(q_1 < e_1, e_2 >)$$

Since a_1 is a fresh variable, it can be removed from the heap. We know that $q_1 < e_1, e_2 >$ is not evaluated, therefore $a \notin fv(q_1 < e_1, e_2 >)$. Then, by the $deref_S$ lemma,

$$deref_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}}}(q_1 < e_1, e_2 >) = deref_S(q_1 < e_1, e_2 >)$$

and

$$\begin{aligned}
&deref_{S,a\mapsto q\ t_1{}_{\mathcal{W}},a_1\mapsto q_1<e_1,e_2>}(a_1)\\
&= deref_{S,a\mapsto q\ t_1{}_{\mathcal{W}}}(q_1 < e_1, e_2 >)\\
&\quad \text{since } a_2 \text{ is a fresh variable, it can be removed from the heap}\\
&= deref_S(q_1 < e_1, e_2 >) \text{ by the } deref_S \text{ lemma, because } a \notin fv(q_1 < e_1, e_2 >)
\end{aligned}$$

So the property holds.

– Given $(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; x\ t)$, we have

$$\frac{S(x) = q_1\ \lambda y : q_2\ P.t'}{(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; x\ t) \rightarrow (S \sim^{q_1} x, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; t'[y \mapsto t])}\ \textit{s-app}_{\mathcal{W}}$$

We also know that

$$\frac{S(x) = q_1\ \lambda y : q_2\ P.t'}{(S, a \mapsto q\ t_{1\mathcal{W}}; x\ t) \to (S \sim^{q_1} x, a \mapsto q\ t_{1\mathcal{W}}; t'[y \mapsto t])}\ \textit{s-app}_{\mathcal{W}}$$

We know that if x occurs unrestricted then it will stay in S but, if x occurs linearly then it will not be used again, thus we can use $S \sim^q x = S$.
Then

$$\begin{aligned} & \mathit{deref}_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}}}(t'[y \mapsto t]) \\ = & \mathit{deref}_S(t'[y \mapsto t][a \mapsto (t_1{}^*)_{\mathcal{W}}]) \text{ by the } \mathit{deref}_S \text{ lemma} \\ & \text{by lemma 8, we know that} \\ = & \mathit{deref}_S(t'[y \mapsto t][a \mapsto t_{1\mathcal{W}}]) \end{aligned}$$

and

$$\mathit{deref}_{S,a\mapsto q\ t_{1\mathcal{W}}}(t'[y \mapsto t]) = \mathit{deref}_S(t'[y \mapsto t][a \mapsto t_{1\mathcal{W}}]) \text{ by the } \mathit{deref}_S \text{ lemma}$$

Since we know that $(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; t'[y \mapsto t]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis,

$$(S, a \mapsto \text{ q } t_{1\mathcal{W}}; t'[y \mapsto t]) \twoheadrightarrow (S_2; v_2)$$

and $\mathit{deref}_{S_2}(v_2) = \mathit{deref}_{S_1}(v_1)$. And the property holds.

– Given $(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; (q_1\ \lambda y : q_2\ P.\ t')\ t)$, we have

$$\begin{aligned} & (S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; (q_1\ \lambda y : q_2\ P.\ t')\ t) \\ \to & (S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_1\ t) \end{aligned}$$

and, by rule $\textit{s-app}_{\mathcal{W}}$

$$\begin{aligned} & (S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}, a_1 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_1\ t) \\ \to & (S \sim^{q_1} a_1, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; t'[y \mapsto t]) \end{aligned}$$

We also know that

$$\begin{aligned} & (S, a \mapsto q\ t_{1\mathcal{W}}; (q_1\ \lambda y : q_2\ P.\ t')\ t) \\ \to & (S, a \mapsto q\ t_{1\mathcal{W}}, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_2\ t) \end{aligned}$$

and, by rule $\textit{s-app}_{\mathcal{W}}$

$$\begin{aligned} & (S, a \mapsto q\ t_{1\mathcal{W}}, a_2 \mapsto q_1\ \lambda y : q_2\ P.\ t'; a_2\ t) \\ \to & (S \sim^{q_1} a_2, a \mapsto q\ t_{1\mathcal{W}}; t'[y \mapsto t]) \end{aligned}$$

Since a_1 and a_2 are fresh variables, they will not be used again, so we can use $S \sim^{q_1} a_1 = S$ and $S \sim^{q_1} a_2 = S$.
Then

$$\begin{aligned} & \mathit{deref}_{S,a\mapsto q\ (t_1{}^*)_{\mathcal{W}}}(t'[y \mapsto t]) \\ = & \mathit{deref}_S(t'[y \mapsto t][a \mapsto (t_1{}^*)_{\mathcal{W}}]) \text{ by the } \mathit{deref}_S \text{ lemma} \\ = & \mathit{deref}_S(t'[y \mapsto t][a \mapsto t_{1\mathcal{W}}]) \text{ by lemma 8} \end{aligned}$$

and

$deref_{S,a\mapsto q\ t_1\mathcal{W}}(t'[y \mapsto t]) = deref_S(t'[y \mapsto t][a \mapsto t_1\mathcal{W}])$ by the $deref_S$ lemma

Since we know that $(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; t'[y \mapsto t]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis, $(S, a \mapsto \text{ q } t_1\mathcal{W}; t'[y \mapsto t]) \twoheadrightarrow (S_2; v_2)$ and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. And the property holds.

– Given $(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; \text{split } x \text{ as } y, z \text{ in } t_2)$, we have, by rule *s-split*$_\mathcal{W}$

$$\begin{aligned}&(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; \text{split } x \text{ as } y, z \text{ in } t_2)\\ \rightarrow\ &(S \sim^{q_1} x, a \mapsto q\ ({t_1}^*)_\mathcal{W}; t_2[y \mapsto e_1][z \mapsto e_2])\end{aligned}$$

where $S(x) = q_1 < e_1, e_2 >$.
We also know that, by rule *s-split*$_\mathcal{W}$

$$\begin{aligned}&(S, a \mapsto q\ t_1\mathcal{W}; \text{split } x \text{ as } y, z \text{ in } t_2)\\ \rightarrow\ &(S \sim^{q_1} x, a \mapsto q\ t_1\mathcal{W}; t_2[y \mapsto e_1][z \mapsto e_2])\end{aligned}$$

where $S(x) = q_1 < e_1, e_2 >$.
We know that if x occurs unrestricted then it will stay in S but, if x occurs linearly then it will not be used again, thus we can use $S \sim^{q_1} x = S$.
Then,

$$\begin{aligned}&deref_{S,a\mapsto q\ ({t_1}^*)_\mathcal{W}}(t_2[y \mapsto e_1][z \mapsto e_2])\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto ({t_1}^*)_\mathcal{W}]) \text{ by the } deref_S \text{ lemma}\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto t_1\mathcal{W}]) \text{ by lemma 8}\end{aligned}$$

and

$$\begin{aligned}&deref_{S,a\mapsto q\ t_1\mathcal{W}}(t_2[y \mapsto e_1][z \mapsto e_2])\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto t_1\mathcal{W}]) \text{ by the } deref_S \text{ lemma}\end{aligned}$$

Since we know that $(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; t_2[y \mapsto e_1][z \mapsto e_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis,

$$(S, a \mapsto \text{ q } t_1\mathcal{W}; t_2[y \mapsto e_1][z \mapsto e_2]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. And the property holds.

– Given $(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; \text{split } (q_1 < e_1, e_2 >) \text{ as } y, z \text{ in } t_2)$, we have

$$\begin{aligned}&(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}; \text{split } (q_1 < e_1, e_2 >) \text{ as } y, z \text{ in } t_2)\\ \rightarrow\ &(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}, a_1 \mapsto q_1 < e_1, e_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t_2)\end{aligned}$$

and, by rule *s-split*$_\mathcal{W}$

$$\begin{aligned}&(S, a \mapsto q\ ({t_1}^*)_\mathcal{W}, a_1 \mapsto q_1 < e_1, e_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t_2)\\ \rightarrow\ &(S \sim^{q_1} a_1, a \mapsto q\ ({t_1}^*)_\mathcal{W}; t_2[y \mapsto e_1][z \mapsto e_2])\end{aligned}$$

We also know that

$$\begin{aligned}&(S, a \mapsto q\ t_{1\mathcal{W}}; \text{split } (q_1 < e_1, e_2 >) \text{ as } y, z \text{ in } t_2)\\ \to\ &(S, a \mapsto q\ t_{1\mathcal{W}}, a_2 \mapsto q_1 < e_1, e_2 >; \text{split } a_2 \text{ as } y, z \text{ in } t_2)\end{aligned}$$

and, by rule *s-split*$_{\mathcal{W}}$

$$\begin{aligned}&(S, a \mapsto q\ t_{1\mathcal{W}}, a_2 \mapsto q_1 < e_1, e_2 >; \text{split } a_1 \text{ as } y, z \text{ in } t_2)\\ \to\ &(S \sim^{q_1} a_2, a \mapsto q\ t_{1\mathcal{W}}; t_2[y \mapsto e_1][z \mapsto e_2])\end{aligned}$$

Since both a_1 and a_2 are fresh variables, they will not be used again, so we can conclude that $S \sim^{q_1} a_1 = S$ and $S \sim^{q_1} a_2 = S$.
Then,

$$\begin{aligned}&deref_{S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}}(t_2[y \mapsto e_1][z \mapsto e_2])\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto (t_1{}^*)_{\mathcal{W}}]) \text{ by the } deref_S \text{ lemma}\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto t_{1\mathcal{W}}]) \text{ by lemma 8}\end{aligned}$$

and

$$\begin{aligned}&deref_{S, a \mapsto q\ t_{1\mathcal{W}}}(t_2[y \mapsto e_1][z \mapsto e_2])\\ =\ &deref_S(t_2[y \mapsto e_1][z \mapsto e_2][a \mapsto t_{1\mathcal{W}}]) \text{ by the } deref_S \text{ lemma}\end{aligned}$$

Since we know that $(S, a \mapsto q\ (t_1{}^*)_{\mathcal{W}}; t_2[y \mapsto e_1][z \mapsto e_2]) \twoheadrightarrow (S_1; v_1)$ then, by induction hypothesis,

$$(S, a \mapsto \text{ q } t_{1\mathcal{W}}; t_2[y \mapsto e_1][z \mapsto e_2]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$. And the property holds.

Theorem 1.
If $(S; M_{\mathcal{W}}) \to_k (S'; N_{\mathcal{W}})$ *then*

$$(S^*_{\mathcal{L}}; (M_{\mathcal{W}})^*_{\mathcal{L}}) \Downarrow (\Gamma; N_{\mathcal{L}}) \text{ and } deref_{\Gamma}(N_{\mathcal{L}}) = deref_{S'^*_{\mathcal{L}}}((N_{\mathcal{W}})^*_{\mathcal{L}})$$

<u>Proof</u> By induction on $(S; t) \to_k (S'; t')$:

- Base case: $(k = 1)$
  - Given $(S; q\ \lambda y : q'\ P.\ t)$, by applying rule *s-abs*$_{\mathcal{W}}$, we have

    $$\frac{}{(S; q\ \lambda y : q'\ P.\ t) \to (S, a \mapsto q\ \lambda y : q'\ P.\ t; a)}$$

    Since $(q\ \lambda y : \text{q}'\ \text{P.}\ t)^*_{\mathcal{L}} = \lambda_\pi y :\ P_{\mathcal{L}}.\ t^*_{\mathcal{L}}$, with $\pi = \text{q}'_{\mathcal{L}}$, then by applying rule *s-abs*$_{\mathcal{L}}$,

    $$\frac{}{(S^*_{\mathcal{L}}; \lambda_\pi y :\ P_{\mathcal{L}}.\ t^*_{\mathcal{L}}) \Downarrow (S^*_{\mathcal{L}}; \lambda_\pi y :\ P_{\mathcal{L}}.\ t^*_{\mathcal{L}})}$$

    Now, we know that

    $$deref_{S^*_{\mathcal{L}}, a:_\pi A = \lambda_\pi y:\ P_{\mathcal{L}}.\ t^*_{\mathcal{L}}}(a) = deref_{S^*_{\mathcal{L}}}(\lambda_\pi y :\ P_{\mathcal{L}}.\ t^*_{\mathcal{L}})$$

    Since a is a fresh variable, it occurs linearly. So, the property holds.

- Given $(S; q < t_1, t_2 >)$, by applying rule *s-pair*$_{\mathcal{W}}$, we know

$$\frac{}{(S; q < t_1, t_2 >) \to (S, a \mapsto q < t_1, t_2 >; a)}$$

Since $(q < t_1, t_2 >)^*_{\mathcal{L}} = \text{let}_\pi\ a_1 : A_1 = t^*_{1\mathcal{L}}, a_2 : A_2 = t^*_{2\mathcal{L}}$ in $< a_1, a_2 >_\pi$, with $\pi = q_{\mathcal{L}}$.
Let $\Gamma = \{S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_1{}^*_{\mathcal{L}}, a_2 :_\pi A_2 = t_2{}^*_{\mathcal{L}}\}$, then by applying rule *s-pair*$_{\mathcal{L}}$

$$\frac{\dfrac{}{(\Gamma; < a_1, a_2 >_\pi) \Downarrow (\Gamma; < a_1, a_2 >_\pi)}\ \textit{s-pair}_{\mathcal{L}}}{(S^*_{\mathcal{L}}; \text{let}_\pi\ a_1 : A_1 = t_1{}^*_{\mathcal{L}}, a_2 : A_2 = t_2{}^*_{\mathcal{L}} \text{ in } < a_1, a_2 >_\pi) \Downarrow (\Gamma; < a_1, a_2 >_\pi)}\ \textit{s-let}_{\mathcal{L}}$$

We now have,

$$\begin{aligned}
& \mathit{deref}_{S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_1{}^*_{\mathcal{L}}, a_2 :_\pi A_2 = t_2{}^*_{\mathcal{L}}}(< a_1, a_2 >_\pi) \\
= \text{let } (v_1, \Gamma_1) &= \mathit{deref}_{S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_1{}^*_{\mathcal{L}}, a_2 :_\pi A_2 = t_2{}^*_{\mathcal{L}}}(a_1) \\
&= \mathit{deref}_{S^*_{\mathcal{L}}, a_2 :_\pi A_2 = t_2{}^*_{\mathcal{L}}}(t_1{}^*_{\mathcal{L}}) \text{ by the } \mathit{deref}_\Gamma \text{ lemma} \\
&\text{since } a_2 \notin fv(t_1{}^*_{\mathcal{L}}) \\
&= \mathit{deref}_{S^*_{\mathcal{L}}}(t_1{}^*_{\mathcal{L}}) \\
(v_2, \Gamma_2) &= \mathit{deref}_{\Gamma_1, a_1 :_\pi A_1 = t_1{}^*_{\mathcal{L}}, a_2 :_\pi A_2 = t_2{}^*_{\mathcal{L}}}(a_2) \\
&= \mathit{deref}_{S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_1{}^*_{\mathcal{L}}}(t_2{}^*_{\mathcal{L}}) \text{ by the } \mathit{deref}_\Gamma \text{ lemma} \\
&\text{since } a_1 \notin fv(t_2{}^*_{\mathcal{L}}) \\
&= \mathit{deref}_{S^*_{\mathcal{L}}}(t_2{}^*_{\mathcal{L}}) \\
\text{in } (< v_1, v_2 >_\pi, \Gamma_2)
\end{aligned}$$

and

$$\mathit{deref}_{S^*_{\mathcal{L}}, a :_\pi A_1 *_\pi A_2 = < t_1{}^*_{\mathcal{L}}, t_2{}^*_{\mathcal{L}} >_\pi}(a) = \mathit{deref}_{S^*_{\mathcal{L}}}(< t_1{}^*_{\mathcal{L}}, t_2{}^*_{\mathcal{L}} >_\pi)$$

a is a fresh variable, so it occurs linearly. Since we already know that $(v_1, \Gamma_1) = \mathit{deref}_{S^*_{\mathcal{L}}}(t_1{}^*_{\mathcal{L}})$ and $(v_2, \Gamma_2) = \mathit{deref}_{S^*_{\mathcal{L}}}(t_2{}^*_{\mathcal{L}})$, the property holds.

– Inductive case:
- Given $(S; x\ t)$, by applying rule *s-app*$_{\mathcal{W}}$, we have

$$\frac{S(x) = q\ \lambda y : q'\ P.\ t_1}{(S; x\ t) \to (S \sim^q x; t_1[y \mapsto t])}$$

where $\mathit{var}(x)$. And we know that $\exists k \in \mathbb{N}$ such that

$$(S \sim^q x; t_1[y \mapsto t]) \to_k (S''; v)$$

Since $(x\ t)^*_{\mathcal{L}} = \text{let}_\pi\ a_1 : A_1 = t^*_{\mathcal{L}}$ in $x\ a_1$, where $\pi = q'_{\mathcal{L}}$, and

$$\{x :_\pi A_2 = \lambda_\pi y : P_{\mathcal{L}}.\ t_1{}^*_{\mathcal{L}}\} \in S^*_{\mathcal{L}}$$

  * q = lin
    Let $\Gamma_1 = \{S^*_{\mathcal{L}} \setminus \{x\}, a_1 :_\pi A_1 = t^*_{\mathcal{L}}\}$ and let us call the following evaluation Ω.

$$\dfrac{\dfrac{}{(\Gamma_1; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L}) \Downarrow (\Gamma_1; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L})}\ \textit{s-abs}_\mathcal{L}}{(S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; x) \Downarrow (\Gamma_1; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L})}\ \textit{s-linvar}_\mathcal{L}$$

and

$$\dfrac{\dfrac{\Omega \qquad (\Gamma_1; t_1{}^*_\mathcal{L}[y \mapsto a_1]) \Downarrow (\Gamma; \sigma_1)}{(S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; x\ a_1) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-app}_\mathcal{L}}{(S^*_\mathcal{L}; \mathrm{let}_\pi\ a_1 : A_1 = t^*_\mathcal{L}\ \mathrm{in}\ x\ a_1) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-let}_\mathcal{L}$$

We know that $(S \setminus \{x\}; t_1[y \mapsto t]) \rightarrow_k (S''; v)$ then, by induction hypothesis, we have

$$(S^*_\mathcal{L} \setminus \{x\}; (t_1[y \mapsto t])^*_\mathcal{L}) \Downarrow (\Delta; \phi)$$

and $deref_\Delta(\phi) = deref_{S''^*_\mathcal{L}}(v^*_\mathcal{L})$, for some Δ and ϕ.
Since we know that $(t_1[y \mapsto t])^*_\mathcal{L} = t_1{}^*_\mathcal{L}[y \mapsto t^*_\mathcal{L}]$ by the substitution lemma and, since a_1 is a fresh variable, $a_1 \notin fv(t_1{}^*_\mathcal{L})$, so by the Γ substitution lemma, if

$$(\Gamma_1; t_1{}^*_\mathcal{L}[y \mapsto a_1]) \Downarrow (\Gamma; \sigma_1)$$

then $(S^*_\mathcal{L} \setminus \{x\}; t_1{}^*_\mathcal{L}[y \mapsto t^*_\mathcal{L}]) \Downarrow (\Gamma; \sigma_1)$ and $deref_\Gamma(\sigma_1) = deref_{S''^*_\mathcal{L}}(v^*_\mathcal{L})$.
And the property holds.

* q = un
Let $\Gamma_2 = \{S^*_\mathcal{L} \setminus \{x\}, a_1 :_\pi A_1 = t^*_\mathcal{L}\}$ and let us call the following the following evaluation Ω.

$$\dfrac{\dfrac{}{(\Gamma_2; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L}) \Downarrow (\Gamma_2; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L})}\ \textit{s-abs}_\mathcal{L}}{(S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; x) \Downarrow (S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; \lambda_\pi y : P_\mathcal{L}.\ t_1{}^*_\mathcal{L})}\ \textit{s-unvar}_\mathcal{L}$$

and

$$\dfrac{\dfrac{\Omega \qquad (S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; t_1{}^*_\mathcal{L}[y \mapsto a_1]) \Downarrow (\Gamma'; \sigma_2)}{(S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; x\ a_1) \Downarrow (\Gamma'; \sigma_2)}\ \textit{s-app}_\mathcal{L}}{(S^*_\mathcal{L}; \mathrm{let}_\pi\ a_1 : A_1 = t^*_\mathcal{L}\ \mathrm{in}\ x\ a_1) \Downarrow (\Gamma'; \sigma_2)}\ \textit{s-let}_\mathcal{L}$$

We know that $(S; t_1[y \mapsto t]) \rightarrow_k (S''; v)$ then, by induction hypothesis, we have

$$(S^*_\mathcal{L}; (t_1[y \mapsto t])^*_\mathcal{L}) \Downarrow (\Delta; \phi)$$

and $deref_\Delta(\phi) = deref_{S''^*_\mathcal{L}}(v^*_\mathcal{L})$, for some Δ and ϕ.
Since we know that $(t_1[y \mapsto t])^*_\mathcal{L} = t_1{}^*_\mathcal{L}[y \mapsto t^*_\mathcal{L}]$ by the substitution lemma and, since a_1 is a fresh variable, $a_1 \notin fv(t_1{}^*_\mathcal{L})$, so by the Γ substitution lemma, if

$$(S^*_\mathcal{L}, a_1 :_\pi A_1 = t^*_\mathcal{L}; t_1{}^*_\mathcal{L}[y \mapsto a_1]) \Downarrow (\Gamma'; \sigma_2)$$

then $(S^*_\mathcal{L}; t_1{}^*_\mathcal{L}[y \mapsto t^*_\mathcal{L}]) \Downarrow (\Gamma'; \sigma_2)$ and $deref_{\Gamma'}(\sigma_2) = deref_{S''^*_\mathcal{L}}(v^*_\mathcal{L})$.
So the property holds.

- Given $(S; t_1\ t_2)$, where $\neg var(t_1)$ and $\neg var(t_2)$, then, by lemma 1.1, $\exists k_1, k_2 \in \mathbb{N}$ such that $k = k_1 + k_2$ and
$(S; t_1\ t_2) \rightarrow_{k_1} (S'; x\ t_2) \rightarrow_{k_2} (S''; v)$. Since, by applying rule *s-app*$_{\mathcal{W}}$, we have
$$\frac{S'(x) = q\ \lambda y : q'\ P.\ \mathrm{t}}{(S'; x\ t_2) \rightarrow (S' \sim^q x; t[y \mapsto t_2])}$$
we can conclude that
$$\frac{\begin{array}{c}(S; t_1\ t_2) \rightarrow_{k_1} (S'; x\ t_2)\\ (S'; x\ t_2) \rightarrow (S' \sim^q x; t[y \mapsto t_2])\\ (S' \sim^q x; t[y \mapsto t_2]) \rightarrow_{k_2-1} (S''; v)\end{array}}{(S; t_1\ t_2) \rightarrow_k (S''; v)}$$
Now we have $(t_1\ t_2)^*_{\mathcal{L}} = \mathrm{let}_\pi\ a_1 : A_1 = t_2{}^*_{\mathcal{L}}$ in $t_1{}^*_{\mathcal{L}}\ a_1$ and, since we know that $(S; t_1) \rightarrow_{k_1} (S'; x)$ then, by induction hypothesis,
$$deref_{S'^*_{\mathcal{L}}}(x) = deref_{S'^*_{\mathcal{L}}}(\lambda_\pi y : P_{\mathcal{L}}.\ t^*_{\mathcal{L}})$$
where $\pi = q'_{\mathcal{L}}$, and $(S^*_{\mathcal{L}}; t_1{}^*_{\mathcal{L}}) \Downarrow (S'^*_{\mathcal{L}}; \lambda_\pi y : P_{\mathcal{L}}.\ t^*_{\mathcal{L}})$. Now we have,
$$\frac{\dfrac{(S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_2{}^*_{\mathcal{L}}; t_1{}^*_{\mathcal{L}}) \Downarrow (S'^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_2{}^*_{\mathcal{L}}; \lambda_\pi y : P_{\mathcal{L}}.\ t^*_{\mathcal{L}})\quad (S'^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_2{}^*_{\mathcal{L}}; t^*_{\mathcal{L}}[y \mapsto a_1]) \Downarrow (\Gamma; \sigma_1)}{(S^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_2{}^*_{\mathcal{L}}; t_1{}^*_{\mathcal{L}}\ a_1) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-app}_{\mathcal{L}}}{(S^*_{\mathcal{L}}; \mathrm{let}_\pi\ a_1 : A_1 = t_2{}^*_{\mathcal{L}} \text{ in } t_1{}^*_{\mathcal{L}}\ a_1) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-let}_{\mathcal{L}}$$
We know that $(S' \sim^q x; t[y \mapsto t_2]) \rightarrow_{k_2-1} (S''; v)$ then, by induction hypothesis, $((S' \sim^q x)^*_{\mathcal{L}}; (t[y \mapsto t_2])^*_{\mathcal{L}}) \Downarrow (\Gamma'; \sigma_2)$, for some Γ' and σ_2, and $deref_{S''^*_{\mathcal{L}}}(v^*_{\mathcal{L}}) = deref_{\Gamma'}(\sigma_2)$. If x occurs unrestricted then it will stay in S but, if x occurs linearly then it will not be used again, thus we can use $S' \sim^q x = S'$.
We know, by the substitution lemma, that $(t[y \mapsto t_2])^*_{\mathcal{L}} = t^*_{\mathcal{L}}[y \mapsto t_2{}^*_{\mathcal{L}}]$ and, by the Γ substitution lemma, if
$$(S'^*_{\mathcal{L}}, a_1 :_\pi A_1 = t_2{}^*_{\mathcal{L}}; t^*_{\mathcal{L}}[y \mapsto a_1]) \Downarrow (\Gamma; \sigma_1)$$
then $(S'^*_{\mathcal{L}}; t^*_{\mathcal{L}}[y \mapsto t_2{}^*_{\mathcal{L}}]) \Downarrow (\Gamma; \sigma_1)$ and $deref_{S''^*_{\mathcal{L}}}(v^*_{\mathcal{L}}) = deref_{\Gamma}(\sigma_1)$. So, the property holds.
- Given (S; split x as y, z in t), by applying rule *s-split*$_{\mathcal{W}}$, we have
$$\frac{\mathrm{S}(x) = \mathrm{q} < t_1, t_2 >}{(\mathrm{S};\ \text{split } x \text{ as } y, z \text{ in t}) \rightarrow (\mathrm{S} \sim^q x;\ \mathrm{t}[y \mapsto t_1][z \mapsto t_2])}$$
where $var(x)$. And we know that $\exists k \in \mathbb{N}$ such that
$$(S \sim^q x; t[y \mapsto t_1][z \mapsto t_2]) \rightarrow_k (S'; v)$$
Since $(\text{split } x \text{ as } y, z \text{ in t})^*_{\mathcal{L}} = \text{split } x \text{ as } y, z \text{ in } t^*_{\mathcal{L}}$ and
$$\{x :_\pi A_1 = \mathrm{let}_\pi\ y_1 : A_2 = t_1{}^*_{\mathcal{L}}, z_1 : A_3 = t_2{}^*_{\mathcal{L}} \text{ in } < y_1, z_1 >_\pi\} \in S^*_{\mathcal{L}}$$
where $\pi = q_{\mathcal{L}}$, then we have two cases:

* q = lin
  Let $\Gamma_1 = \{S^*_{\mathcal{L}} \setminus \{x\}, y_1 :_\pi A_2 = t_1{}^*_{\mathcal{L}}, z_1 :_\pi A_3 = t_2{}^*_{\mathcal{L}}\}$ and let us consider Ω.

$$\dfrac{\dfrac{\dfrac{}{(\Gamma_1; < y_1, z_1 >_\pi) \Downarrow (\Gamma_1; < y_1, z_1 >_\pi)}\ \textit{s-pair}_{\mathcal{L}}}{(S^*_{\mathcal{L}} \setminus \{x\}; \text{let}_\pi\ y_1 : A_2 = t_1{}^*_{\mathcal{L}}, z_1 : A_3 = t_2{}^*_{\mathcal{L}} \text{ in } < y_1, z_1 >_\pi) \Downarrow (\Gamma_1; < y_1, z_1 >_\pi)}\ \textit{s-let}_{\mathcal{L}}}{(S^*_{\mathcal{L}}; x) \Downarrow (\Gamma_1; < y_1, z_1 >_\pi)}\ \textit{s-linvar}_{\mathcal{L}}$$

  and

$$\dfrac{\Omega \qquad (\Gamma_1; t^*_{\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma; \sigma_1)}{(S^*_{\mathcal{L}}; \text{split } x \text{ as } y, z \text{ in } t^*_{\mathcal{L}}) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-split}_{\mathcal{L}}$$

  We know that $(S \setminus \{x\}; t[y \mapsto t_1][z \mapsto t_2]) \rightarrow_k (S'; v)$ then, by induction hypothesis, we have

$$(S^*_{\mathcal{L}} \setminus \{x\}; (t[y \mapsto t_1][z \mapsto t_2])^*_{\mathcal{L}}) \Downarrow (\Delta; \phi)$$

  and $deref_\Delta(\phi) = deref_{S'^*_{\mathcal{L}}}(v^*_{\mathcal{L}})$, for some Δ and ϕ.
  Since we know that $(t[y \mapsto t_1][z \mapsto t_2])^*_{\mathcal{L}} = t^*_{\mathcal{L}}[y \mapsto t_1{}^*_{\mathcal{L}}][z \mapsto t_2{}^*_{\mathcal{L}}]$ by the substitution lemma and, since y_1 and z_1 are fresh variables, $y_1 \notin fv(t^*_{\mathcal{L}})$ and $z_1 \notin fv(t^*_{\mathcal{L}})$, so by the Γ substitution lemma, if

$$(\Gamma_1; t^*_{\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma; \sigma_1)$$

  then

$$(S^*_{\mathcal{L}} \setminus \{x\}; t^*_{\mathcal{L}}[y \mapsto t_1{}^*_{\mathcal{L}}][z \mapsto t_2{}^*_{\mathcal{L}}]) \Downarrow (\Gamma; \sigma_1)$$

  and $deref_\Gamma(\sigma_1) = deref_{S'^*_{\mathcal{L}}}(v^*_{\mathcal{L}})$. And the property holds.
* q = un
  Let $\Gamma_2 = \{S^*_{\mathcal{L}} \setminus \{x\}, y_1 :_\pi A_2 = t_1{}^*_{\mathcal{L}}, z_1 :_\pi A_3 = t_2{}^*_{\mathcal{L}}\}$ and let us consider the following evaluation Ω.

$$\dfrac{\dfrac{\dfrac{}{(\Gamma_2; < y_1, z_1 >_\pi) \Downarrow (\Gamma_2; < y_1, z_1 >_\pi)}\ \textit{s-pair}_{\mathcal{L}}}{(S^*_{\mathcal{L}} \setminus \{x\}; \text{let}_\pi\ y_1 : A_2 = t_1{}^*_{\mathcal{L}}, z_1 : A_3 = t_2{}^*_{\mathcal{L}} \text{ in } < y_1, z_1 >_\pi) \Downarrow (\Gamma_2; < y_1, z_1 >_\pi)}\ \textit{s-let}_{\mathcal{L}}}{(S^*_{\mathcal{L}}; x) \Downarrow (S^*_{\mathcal{L}}, y_1 :_\pi A_2 = t_1{}^*_{\mathcal{L}}, z_1 :_\pi A_3 = t_2{}^*_{\mathcal{L}}; < y_1, z_1 >_\pi)}\ \textit{s-unvar}_{\mathcal{L}}$$

  and

$$\dfrac{\Omega \qquad (S^*_{\mathcal{L}}, y_1 :_\pi A_2 = t_1{}^*_{\mathcal{L}}, z_1 :_\pi A_3 = t_2{}^*_{\mathcal{L}}; t^*_{\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma; \sigma_1)}{(S^*_{\mathcal{L}}; \text{split } x \text{ as } y, z \text{ in } t^*_{\mathcal{L}}) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-split}_{\mathcal{L}}$$

  We know that (S; t$[y \mapsto t_1][z \mapsto t_2]) \rightarrow_k$ (S′; v) then, by induction hypothesis, we have

$$(S^*_{\mathcal{L}}; (t[y \mapsto t_1][z \mapsto t_2])^*_{\mathcal{L}}) \Downarrow (\Delta; \phi)$$

  and $deref_\Delta(\phi) = deref_{S'^*_{\mathcal{L}}}(v^*_{\mathcal{L}})$, for some Δ and ϕ.

Since we know that $(\mathrm{t}[y \mapsto t_1][z \mapsto t_2])^*_{\mathcal{L}} = t^*_{\mathcal{L}}[y \mapsto {t_1}^*_{\mathcal{L}}][z \mapsto {t_2}^*_{\mathcal{L}}]$ by the substitution lemma and, since y_1 and z_1 are fresh variables, $y_1 \notin fv(t^*_{\mathcal{L}})$ and $z_1 \notin fv(t^*_{\mathcal{L}})$, so by the Γ substitution lemma, if $(\mathrm{S}^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_1}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_2}^*_{\mathcal{L}};\ t^*_{\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma'; \sigma_2)$ then

$$(S^*_{\mathcal{L}}; t^*_{\mathcal{L}}[y \mapsto {t_1}^*_{\mathcal{L}}][z \mapsto {t_2}^*_{\mathcal{L}}])^*_{\mathcal{L}}) \Downarrow (\Gamma'; \sigma_2)$$

and $deref_{\Gamma'}(\sigma_2) = deref_{S'^*_{\mathcal{L}}}(v^*_{\mathcal{L}})$. And the property holds.

- Given (S; split t_1 as y, z in t_2), where $\neg var(t_1)$, then, by lemma 1.2, $\exists k_1, k_2 \in \mathbb{N}$ such that $k = k_1 + k_2$ and
(S; split t_1 as y, z in t_2) $\rightarrow_{k_1}$ (S′; split x as y, z in t_2) $\rightarrow_{k_2}$ (S″; v). Since, by applying rule *s-split*$_{\mathcal{W}}$, we have

$$\frac{\mathrm{S}'(x) = \mathrm{q} < t_3, t_4 >}{(\mathrm{S}';\ \text{split } x \text{ as } y, z \text{ in } t_2) \rightarrow (\mathrm{S}' \sim^q x;\ \mathrm{t}[y \mapsto t_3][z \mapsto t_4])}$$

we can conclude that

$$\frac{\begin{array}{c}(\mathrm{S};\ \text{split } t_1 \text{ as } y, z \text{ in } t_2) \rightarrow_{k_1} (\mathrm{S}';\ \text{split } x \text{ as } y, z \text{ in } t_2) \\ (\mathrm{S}';\ \text{split } x \text{ as } y, z \text{ in } t_2) \rightarrow (\mathrm{S}' \sim^q x;\ \mathrm{t}[y \mapsto t_3][z \mapsto t_4]) \\ (\mathrm{S}' \sim^q x;\ \mathrm{t}[y \mapsto t_3][z \mapsto t_4]) \rightarrow_{k_2-1} (\mathrm{S}'';\ v)\end{array}}{(\mathrm{S};\ \text{split } t_1 \text{ as } y, z \text{ in } t_2) \rightarrow_k (\mathrm{S}'';\ v)}$$

Now we have $(\text{split } t_1 \text{ as } y, z \text{ in } t_2)^*_{\mathcal{L}} = \text{split } {t_1}^*_{\mathcal{L}} \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}$ and, since we know that (S; t_1) $\rightarrow_{k_1}$ (S′; x) then, by induction hypothesis,

$$\begin{aligned} deref_{S'^*_{\mathcal{L}}}(x) &= deref_{S'^*_{\mathcal{L}}}(\mathrm{let}_\pi\ y_1 : A_2 = {t_3}^*_{\mathcal{L}}, z_1 : A_3 = {t_4}^*_{\mathcal{L}} \text{ in } < y_1, z_1 >_\pi) \\ &= deref_{S'^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_3}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_4}^*_{\mathcal{L}}}(< y_1, z_1 >_\pi) \end{aligned}$$

where $\pi = \mathrm{q}_{\mathcal{L}}$, and $(\mathrm{S}^*_{\mathcal{L}};\ {t_1}^*_{\mathcal{L}}) \Downarrow (\mathrm{S}'^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_3}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_4}^*_{\mathcal{L}};\ < y_1, z_1 >_\pi)$. Now we have,

$$\frac{\begin{array}{c}(S^*_{\mathcal{L}}; {t_1}^*_{\mathcal{L}}) \Downarrow (S'^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_3}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_4}^*_{\mathcal{L}}; < y_1, z_1 >_\pi) \\ (S'^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_3}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_4}^*_{\mathcal{L}}; t^*_{2\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma; \sigma_1)\end{array}}{(S^*_{\mathcal{L}}; \text{split } {t_1}^*_{\mathcal{L}} \text{ as } y, z \text{ in } {t_2}^*_{\mathcal{L}}) \Downarrow (\Gamma; \sigma_1)}\ \textit{s-split}_{\mathcal{L}}$$

We know that $(S' \sim^q x; t_2[y \mapsto t_3][z \mapsto t_4]) \rightarrow_{k_2-1} (S''; v)$ then, by induction hypothesis,

$$((S' \sim^q x)^*_{\mathcal{L}}; (t_2[y \mapsto t_3][z \mapsto t_4])^*_{\mathcal{L}}) \Downarrow (\Gamma'; \sigma_2)$$

for some Γ' and σ_2, and $deref_{S''^*_{\mathcal{L}}}(v^*_{\mathcal{L}}) = deref_{\Gamma'}(\sigma_2)$. If x occurs unrestricted then it will stay in S but, if x occurs linearly then it will not be used again, thus we can use $S' \sim^q x = S'$.
We know, by the substitution lemma, that $(t_2[y \mapsto t_3][z \mapsto t_4])^*_{\mathcal{L}} = {t_2}^*_{\mathcal{L}}[y \mapsto {t_3}^*_{\mathcal{L}}][z \mapsto {t_4}^*_{\mathcal{L}}]$ and, by the Γ substitution lemma, if

$$(S'^*_{\mathcal{L}}, y_1 :_\pi A_2 = {t_3}^*_{\mathcal{L}}, z_1 :_\pi A_3 = {t_4}^*_{\mathcal{L}}; t^*_{2\mathcal{L}}[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Gamma; \sigma_1)$$

then $(S'^*_{\mathcal{L}}; t^*_{2\mathcal{L}}[y \mapsto {t_3}^*_{\mathcal{L}}][z \mapsto {t_4}^*_{\mathcal{L}}]) \Downarrow (\Gamma; \sigma_1)$ and $deref_{S''^*_{\mathcal{L}}}(v^*_{\mathcal{L}}) = deref_{\Gamma}(\sigma_1)$. So, the property holds.

Theorem 2.
If $(\Gamma; M^*_{\mathcal{L}}) \Downarrow (\Gamma'; N_{\mathcal{L}})$ *then*

$$(\Gamma_{\mathcal{W}}; (M_{\mathcal{L}})_{\mathcal{W}}) \twoheadrightarrow (S; N_{\mathcal{W}}) \text{ and } deref_{\mathcal{S}}(N_{\mathcal{W}}) = deref_{\Gamma'_{\mathcal{W}}}((N_{\mathcal{L}})_{\mathcal{W}})$$

<u>Proof</u> By induction on $(\Gamma; t) \Downarrow (\Gamma'; t')$:

– Base case:
- Given $(\Gamma; (\lambda_\pi y : P.\ t)^*) = (\Gamma; \lambda_\pi y : P.\ t^*)$, we have

$$\frac{}{(\Gamma; \lambda_\pi y : P.\ t^*) \Downarrow (\Gamma; \lambda_\pi y : P.\ t^*)}\ \textit{s-abs}_{\mathcal{L}}$$

Now, we know that $(\Gamma_{\mathcal{W}}; (\lambda_\pi y : P.\ t)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; \pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}})$.
By rule *s-abs*$_{\mathcal{W}}$,

$$\frac{}{(\Gamma_{\mathcal{W}}; \pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}}) \to (\Gamma_{\mathcal{W}}, a \mapsto \pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}}; a)}$$

and

$$\begin{aligned} deref_{\Gamma_{\mathcal{W}}, a \mapsto \pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}}}(a) &= deref_{\Gamma_{\mathcal{W}}}(\pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ t_{\mathcal{W}}) \\ &= \text{let } (v, S) = deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}) \\ &\quad \text{in } (\pi_{\mathcal{W}}\ \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ v, S) \end{aligned}$$

Since a is a fresh variable, it occurs linearly.
We know that, by lemma 9, $deref_{\Gamma_{\mathcal{W}}}(t_{\mathcal{W}}) = deref_{\Gamma_{\mathcal{W}}}(t^*{}_{\mathcal{W}})$, so the property holds.
- Given

$$(\Gamma; (< t_1, t_2 >_\pi)^*) = (\Gamma; \text{let}_\pi\ a_1 : A_1 = {t_1}^*, a_2 : A_2 = {t_2}^*\ \text{in}\ < a_1, a_2 >_\pi)$$

Let $\Gamma_1 = \{\Gamma, a_1 :_\pi A_1 = {t_1}^*, a_2 :_\pi A_2 = {t_2}^*\}$, we have

$$\frac{\dfrac{}{(\Gamma_1; < a_1, a_2 >_\pi) \Downarrow (\Gamma_1; < a_1, a_2 >_\pi)}\ \textit{s-pair}_{\mathcal{L}}}{(\Gamma; \text{let}_\pi\ a_1 : A_1 = {t_1}^*, a_2 : A_2 = {t_2}^*\ \text{in}\ < a_1, a_2 >_\pi) \Downarrow (\Gamma_1; < a_1, a_2 >_\pi)}\ \textit{s-let}_{\mathcal{L}}$$

Now, we know that $(\Gamma_{\mathcal{W}}; (< t_1, t_2 >_\pi)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; \pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >)$.
By rule *s-pair*$_{\mathcal{W}}$,

$$\frac{}{(\Gamma_{\mathcal{W}}; \pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >) \to (\Gamma_{\mathcal{W}}, a \mapsto \pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >; a)}$$

and

$$\begin{aligned} deref_{\Gamma_{\mathcal{W}}, a \mapsto \pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >}(a) &= deref_{\Gamma_{\mathcal{W}}}(\pi_{\mathcal{W}} < t_{1\mathcal{W}}, t_{2\mathcal{W}} >) \\ &= \text{let } (v_1, S_1) = deref_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}}) \\ &\qquad (v_2, S_2) = deref_{S_1}(t_{2\mathcal{W}}) \\ &\quad \text{in } (\pi_{\mathcal{W}} < v_1, v_2 >, S_2) \end{aligned}$$

Since a is a fresh variable, it occurs linearly.

And we have

$$
\begin{aligned}
& \mathit{deref}_{\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}({t_1}^*)_{\mathcal{W}}, a_2 \mapsto \pi_{\mathcal{W}}({t_2}^*)_{\mathcal{W}}}((< a_1, a_2 >_\pi)_{\mathcal{W}}) \\
= \; & \mathit{deref}_{\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}({t_1}^*)_{\mathcal{W}}, a_2 \mapsto \pi_{\mathcal{W}}({t_2}^*)_{\mathcal{W}}}(\pi_{\mathcal{W}} \; < a_1, a_2 >) \\
& \text{since } a_1 \text{ and } a_2 \text{ are fresh variables, they can be removed from the heap} \\
= \; & \mathit{deref}_{\Gamma_{\mathcal{W}}}(\pi_{\mathcal{W}} < ({t_1}^*)_{\mathcal{W}}, ({t_2}^*)_{\mathcal{W}} >) \\
= \; & \text{let } ({v_1}', {S_1}') = \mathit{deref}_{\Gamma_{\mathcal{W}}}(({t_1}^*)_{\mathcal{W}}) \\
& \quad\;\; ({v_2}', {S_2}') = \mathit{deref}_{S_2}(({t_2}^*)_{\mathcal{W}}) \\
& \text{in } (\pi_{\mathcal{W}} < {v_1}', {v_2}' >, {S_2}')
\end{aligned}
$$

By lemma 9, we know that $\mathit{deref}_{\Gamma_{\mathcal{W}}}(({t_1}^*)_{\mathcal{W}}) = \mathit{deref}_{\Gamma_{\mathcal{W}}}(t_{1\mathcal{W}})$, which means that $S_1 = {S_1}'$, then we also know, by lemma 9, that

$$\mathit{deref}_{{S_1}'}(({t_2}^*)_{\mathcal{W}}) = \mathit{deref}_{S_1}(t_{2\mathcal{W}})$$

And the property holds.

– Inductive case:
  - Given $(\Gamma, x :_\pi A = t; x^*) = (\Gamma, x :_\pi A = t; x)$, there are two cases:
    * $\pi = 1$
      We have
      $$\frac{(\Gamma; t) \Downarrow (\Delta; \sigma)}{(\Gamma, x :_1 A = t; x) \Downarrow (\Delta; \sigma)} \; \textit{s-linvar}_{\mathcal{L}}$$
      Since we know that $(\Gamma; t) \Downarrow (\Delta; \sigma)$ then, by induction hypothesis, $(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S; v)$ and $\mathit{deref}_S(v) = \mathit{deref}_{\Delta_{\mathcal{W}}}(\sigma_{\mathcal{W}})$. So the property holds.
    * $\pi = \omega$
      We have
      $$\frac{(\Gamma; t) \Downarrow (\Delta; \sigma)}{(\Gamma, x :_\omega A = t; x) \Downarrow (\Delta, x :_\omega A = \sigma; \sigma)} \; \textit{s-unvar}_{\mathcal{L}}$$
      Since we know that $(\Gamma; t) \Downarrow (\Delta; \sigma)$ then, by induction hypothesis, $(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S; v)$ and $\mathit{deref}_S(v) = \mathit{deref}_{\Delta_{\mathcal{W}}}(\sigma_{\mathcal{W}})$. We know that $x \notin fv(\sigma)$, so we can conclude that
      $$\begin{aligned} \mathit{deref}_{\Delta_{\mathcal{W}}, x \mapsto \text{ un } \sigma_{\mathcal{W}}}(\sigma_{\mathcal{W}}) &= \mathit{deref}_{\Delta_{\mathcal{W}}}(\sigma_{\mathcal{W}}) \\ &= \mathit{deref}_S(v) \end{aligned}$$
      So the property holds.
  - Given $(\Gamma; (t\ x)^*) = (\Gamma; t^*\ x)$, we have
    $$\frac{(\Gamma; t^*) \Downarrow (\Delta; \lambda_\pi y_1 : P.t'^*) \qquad (\Delta; t'^*[y_1 \mapsto x]) \Downarrow (\Theta; \sigma)}{(\Gamma; t^*\ x) \Downarrow (\Theta; \sigma)} \; \textit{s-app}_{\mathcal{W}}$$

  Now, we have

$$
\begin{aligned}
(\Gamma_{\mathcal{W}}; (t\ x)_{\mathcal{W}}) &= (\Gamma_{\mathcal{W}}; t_{\mathcal{W}}\ x_{\mathcal{W}}) \\
&= (\Gamma_{\mathcal{W}}; t_{\mathcal{W}}\ x)
\end{aligned}
$$

Since we know that $(\Gamma; t^*) \Downarrow (\Delta; \lambda_\pi y_1 : P.t'^*)$ then, by induction hypothesis, $(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S; \pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e)$ and, since we know that

$$(\Delta_{\mathcal{W}}; (\lambda_\pi y_2 : P.t'^*)_{\mathcal{W}}) = (\Delta_{\mathcal{W}}; \pi_{\mathcal{W}} \lambda y : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.(t'^*)_{\mathcal{W}})$$

then

$$deref_S(\pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e) = deref_{\Delta_{\mathcal{W}}}(\pi_{\mathcal{W}}\ \lambda y_1 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.(t'^*)_{\mathcal{W}})$$

We know that $(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}\ x) \twoheadrightarrow (S; (\pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e)\ x)$.
We also know that, by rule $s\text{-}abs_{\mathcal{W}}$

$$\frac{}{(S; \pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e) \rightarrow (S, a \mapsto \pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; a)}$$

which means that

$$(S; (\pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e)\ x) \rightarrow (S, a \mapsto \pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; a\ x)$$

Since a is a fresh variable, we can remove it from the heap, then we have

$$\frac{}{(S, a \mapsto \pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; a\ x) \rightarrow (S; e[y_2 \mapsto x])}$$

We know that $(\Delta; t'^*[y_1 \mapsto x]) \Downarrow (\Theta; \sigma)$ and we also know that

$$deref_S(e) = deref_{\Delta_{\mathcal{W}}}((t'^*)_{\mathcal{W}})$$

since, by the substitution lemma, $t'^*[y_1 \mapsto x] = (t'[y_1 \mapsto x])^*$, then, by induction hypothesis, we have that, given $(\Delta; (t'[y_1 \mapsto x])^*) \Downarrow (\Theta; \sigma)$, we know, by α-conversion,

$$(S; e[y_2 \mapsto x]) \twoheadrightarrow (S'; v)$$

and $deref_{S'}(v) = deref_{\Theta_{\mathcal{W}}}(\sigma_{\mathcal{W}})$. And the property holds.

- Given $(\Gamma; (t_1\ t_2)^*) = (\Gamma; \mathrm{let}_\pi\ a : A = {t_2}^*\ \mathrm{in}\ {t_1}^*\ a)$, we have

$$\dfrac{\dfrac{(\Gamma, a :_\pi A = {t_2}^*; {t_1}^*) \Downarrow (\Delta; \lambda_\pi y_1 : P.{t_1}'^*) \qquad (\Delta; {t_1}'^*[y_1 \mapsto a]) \Downarrow (\Theta; \sigma)}{(\Gamma, a :_\pi A = {t_2}^*; {t_1}^*\ a) \Downarrow (\Theta; \sigma)}\ s\text{-}app_{\mathcal{L}}}{(\Gamma; \mathrm{let}_\pi\ a : A = {t_2}^*\ \mathrm{in}\ {t_1}^*\ a) \Downarrow (\Theta; \sigma)}\ s\text{-}let_{\mathcal{L}}$$

Now we have $(\Gamma_{\mathcal{W}}; (t_1\ t_2)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; t_{1\mathcal{W}}\ t_{2\mathcal{W}})$. By proposition 2, we know that, because $a \notin fv({t_1}^*)$, $(\Gamma; {t_1}^*) \Downarrow (\Delta; \lambda_\pi y_1 : P.{t_1}'^*)$ then, by induction hypothesis,

$$(\Gamma_{\mathcal{W}}; t_{1\mathcal{W}}) \twoheadrightarrow (S; \pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ A.\ e)$$

and $deref_S(\pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e) = deref_{\Delta_{\mathcal{W}}}(\pi_{\mathcal{W}}\ \lambda y_1 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ (t_1'^*)_{\mathcal{W}})$.
This means that $(\Gamma_{\mathcal{W}}; t_{1\mathcal{W}}\ t_{2\mathcal{W}}) \twoheadrightarrow (S; (\pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.e)\ t_{2\mathcal{W}})$.
Now we have, by rule $s\text{-}abs_{\mathcal{W}}$,

$$\frac{}{(S; (\pi_{\mathcal{W}} \lambda y_2 : \pi_{\mathcal{W}}\ P.e) \rightarrow (S, a_1 \mapsto \pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; a_1)}$$

So we have $(S; (\pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e)\ t_{2\mathcal{W}}) \rightarrow (S, a_1 \mapsto \pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; a_1\ t_{2\mathcal{W}})$. By rule $s\text{-}app_{\mathcal{W}}$,

$$\frac{}{(S, a_1 \mapsto \pi_{\mathcal{W}}\ \lambda y_2 : \pi_{\mathcal{W}}\ P_{\mathcal{W}}.\ e; z\ t_{2\mathcal{W}}) \to (S \sim^{\pi_{\mathcal{W}}} a_1; e[y_2 \mapsto t_{2\mathcal{W}}])}$$

Since a_1 is a fresh variable, it occurs linearly, so we can use

$$S \sim^{\pi_{\mathcal{W}}} a_1 = S$$

We know that $(\Delta; {t_1}'^{*}[y_1 \mapsto a]) \Downarrow (\Theta; \sigma)$, we have $deref_S(e) = deref_{\Delta_{\mathcal{W}}}((t_1'^{*})_{\mathcal{W}})$ and, by the substitution lemma, ${t_1}'^{*}[y_1 \mapsto a] = ({t_1}'[y_1 \mapsto a])^*$. We also know that, since a is a fresh variable then $a \notin fv(({t_1}'[y_1 \mapsto a])^*)$, so by the Γ substitution lemma, if $(\Delta; ({t_1}'[y_1 \mapsto a])^*) \Downarrow (\Theta; \sigma)$ then

$$(\Delta \setminus \{a\}; ({t_1}'[y_1 \mapsto t_2])^*) \Downarrow (\Theta'; \sigma')$$

and $deref_{\Theta}(\sigma) = deref_{\Theta'}(\sigma')$.
We know that $a \notin fv(\mathrm{e})$, so if $(\Delta \setminus \{a\}; ({t_1}'[y_1 \mapsto t_2])^*) \Downarrow (\Theta'; \sigma')$ then, by induction hypothesis, we have that, by α-conversion,

$$(S; e[y_2 \mapsto t_{2\mathcal{W}}]) \twoheadrightarrow (S'; v)$$

and $deref_{S'}(v) = deref_{\Theta_{\mathcal{W}}}(\sigma_{\mathcal{W}})$. And the property holds.

- Given $(\Gamma;\ (\text{split } t_1 \text{ as } y, z \text{ in } t_2)^*) = (\Gamma;\ \text{split } t_1^* \text{ as } y, z \text{ in } t_2^*)$, we have

$$\frac{(\Gamma;\ t_1^*) \Downarrow (\Delta;\ < y_1, z_1 >_{\pi}) \qquad (\Delta;\ t_2^*[y \mapsto y_1][z \mapsto z_1]) \Downarrow (\Theta;\ z)}{(\Gamma;\ \text{split } t_1^* \text{ as } y, z \text{ in } t_2^*) \Downarrow (\Theta;\ z)}\ \textit{s-split}_{\mathcal{L}}$$

Now we have

$$(\Gamma_{\mathcal{W}}; (\text{split } t_1 \text{ as } y, z \text{ in } t_2)_{\mathcal{W}}) = (\Gamma_{\mathcal{W}}; \text{split } t_{1\mathcal{W}} \text{ as } y, z \text{ in } t_{2\mathcal{W}})$$

We know that $(\Gamma; {t_1}^*) \Downarrow (\Delta; < y_1, z_1 >_{\pi})$ then, by induction hypothesis,

$$(\Gamma_{\mathcal{W}}; t_{1\mathcal{W}}) \twoheadrightarrow (S; \pi_{\mathcal{W}} < e_1, e_2 >)$$

and $deref_S(\pi_{\mathcal{W}} < e_1, e_2 >) = deref_{\Delta_{\mathcal{W}}}(\pi_{\mathcal{W}} < y_1, z_1 >)$.
This means that

$$(\Gamma_{\mathcal{W}}; \text{split } t_{1\mathcal{W}} \text{ as } y, z \text{ in } t_{2\mathcal{W}}) \twoheadrightarrow (S; \text{split } \pi_{\mathcal{W}} < e_1, e_2 > \text{ as } y, z \text{ in } t_{2\mathcal{W}})$$

Now we have, by rule $\textit{s-pair}_{\mathcal{W}}$,

$$\frac{}{(S; \pi_{\mathcal{W}} < e_1, e_2 >) \to (S, a \mapsto \pi_{\mathcal{W}} < e_1, e_2 >; a)}$$

So we have

$$\begin{aligned} &(S; \text{split } \pi_{\mathcal{W}} < e_1, e_2 > \text{ as } y, z \text{ in } t_{2\mathcal{W}}) \\ \to\ &(S, a \mapsto \pi_{\mathcal{W}} < e_1, e_2 >; \text{split } a \text{ as } y, z \text{ in } t_{2\mathcal{W}}) \end{aligned}$$

By rule $\textit{s-split}_{\mathcal{W}}$,

$$\begin{aligned} &(S, a \mapsto \pi_{\mathcal{W}} < e_1, e_2 >; \text{split } a \text{ as } y, z \text{ in } t_{2\mathcal{W}})) \\ \to\ &(S \sim^{\pi_{\mathcal{W}}} a; t_{2\mathcal{W}}[y \mapsto e_1][z \mapsto e_2]) \end{aligned}$$

We know that a is a fresh variable, so it will not be used again, therefore we can use $S \sim^{\pi_{\mathcal{W}}} a = S$.
We know that, by the substitution lemma,

$$t_2{}^*[y \mapsto y_1][z \mapsto z_1] = (t_2[y \mapsto y_1][z \mapsto z_1])^*$$

Then, since $(\Delta; (t_2[y \mapsto y_1][z \mapsto z_1])^*) \Downarrow (\Theta; \sigma)$ then, by induction hypothesis,

$$(\Delta_{\mathcal{W}}; (t_2[y \mapsto y_1][z \mapsto z_1])_{\mathcal{W}}) \twoheadrightarrow (S'; v')$$

and $deref_{\Theta_{\mathcal{W}}}(\sigma_{\mathcal{W}}) = deref_{S'}(v')$.
Since we know that $deref_S(\pi_{\mathcal{W}} < e_1, e_2 >) = deref_{\Delta_{\mathcal{W}}}(\pi_{\mathcal{W}} < y_1, z_1 >)$, the property holds.

- Given

$$\begin{aligned}&(\Gamma; (\text{let}_\pi\ a_1 : A_1 = t_1, \ldots, a_n : A_n = t_n \text{ in } t)^*)\\ =\ &(\Gamma; \text{let}_\pi\ a_1 : A_1 = t_1{}^*, \ldots, a_n : A_n = t_n{}^* \text{ in } t^*)\end{aligned}$$

we have

$$\frac{(\Gamma, a_1 :_\pi A_1 = t_1{}^*, \ldots, a_n :_\pi A_n = t_n{}^*; t^*) \Downarrow (\Delta; \sigma)}{(\Gamma; \text{let}_\pi\ a_1 : A_1 = t_1{}^*, \ldots, a_n : A_n = t_n{}^* \text{ in } t^*) \Downarrow (\Delta; \sigma)}\ \textit{s-let}_{\mathcal{L}}$$

Now we have

$$\begin{aligned}&(\Gamma_{\mathcal{W}}; (\text{let}_\pi\ a_1 : A_1 = t_1, \ldots, a_n : A_n = t_n \text{ in } t)_{\mathcal{W}})\\ =\ &(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}[a_1 \mapsto t_{1\mathcal{W}}] \ldots [a_n \mapsto t_{n\mathcal{W}}])\end{aligned}$$

We know that if $(\Gamma, a_1 :_\pi A_1 = t_1{}^*, \ldots, a_n :_\pi A_n = t_n{}^*; t^*) \Downarrow (\Delta; \sigma)$ then, by induction hypothesis,

$$(\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}(t_1{}^*)_{\mathcal{W}}, \ldots, a_n \mapsto \pi_{\mathcal{W}}(t_n{}^*)_{\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S; v)$$

and $deref_S(v) = deref_{\Delta_{\mathcal{W}}}(\sigma_{\mathcal{W}})$.
By lemma 9, if $(\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}(t_1{}^*)_{\mathcal{W}}, \ldots, a_n \mapsto \pi_{\mathcal{W}}(t_n{}^*)_{\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S; v)$ then

$$(\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}\ t_{1\mathcal{W}}, \ldots, a_n \mapsto \pi_{\mathcal{W}}\ t_{n\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S_1; v_1)$$

and $deref_{S_1}(v_1) = deref_S(v)$. And by S substitution lemma, we know that if $(\Gamma_{\mathcal{W}}, a_1 \mapsto \pi_{\mathcal{W}}\ t_{1\mathcal{W}}, \ldots, a_n \mapsto \pi_{\mathcal{W}}\ t_{n\mathcal{W}}; t_{\mathcal{W}}) \twoheadrightarrow (S_1; v_1)$ then

$$(\Gamma_{\mathcal{W}}; t_{\mathcal{W}}[a_1 \mapsto t_{1\mathcal{W}}] \ldots [a_n \mapsto t_{n\mathcal{W}}]) \twoheadrightarrow (S_2; v_2)$$

and $deref_{S_2}(v_2) = deref_{S_1}(v_1)$.
We can then conclude that $deref_{\Delta_{\mathcal{W}}}(\sigma_{\mathcal{W}}) = deref_{S_2}(v_2)$, so the property holds.